

Edu-Bridge: Technical Documentation

Architecture, database, API, status and handover guide

2026-10-05

Contents

- [Edu-Bridge Documentation](#)
- [Product Overview](#)
- [Architecture](#)
- [Key Flows](#)
- [Setup and Deployment](#)
- [Security Design](#)
- [Current Project Status](#)
- [Remediation Plan](#)
- [Proposed Roadmap](#)
- [Handover Checklist](#)
- [Database Dictionary](#)
- [API Reference](#)

Edu-Bridge Documentation

Complete documentation of the project (Laravel backend + Flutter app) for handover, sale, or onboarding a new developer.

Last reviewed: 2026-10-05 | Arabic version: [../ar/](#)

Where do I start?

You are...	Read in this order
Buyer / evaluator	01 Overview -> 08 Project status -> 09 Roadmap -> 10 Handover checklist
New developer	02 Architecture -> 06 Setup & deploy -> 05 Key flows -> 03 Database -> 04 API reference
Security reviewer	07 Security design -> 08 Project status

Index

#	File	Contents
01	product-overview	What the product is, roles, features
02	architecture	Architecture, layers, external services, Flutter app
03	database	Data dictionary (65 tables) + ERD (<i>auto-generated</i>)
04	api-reference	330 API endpoints grouped by role (<i>auto-generated</i>)
05	key-flows	Diagrams: registration, login, attendance, warnings, leave, chat
06	setup-and-deploy	Install, environment variables, bot, Flutter, deployment
07	security	Existing security controls and developer guidance
08	project-status	The honest state: vulnerabilities, bugs, technical debt
09	roadmap	What to do, in order
10	handover-checklist	Handover checklist, licenses, ownership
11	remediation-plan	The remediation plan and what was executed

Regenerating the auto-generated files

`03-database.md` and `04-api-reference.md` are generated from code, not written by hand:

- **Database:** run `php artisan migrate` on an empty database, then read `information_schema`.
- **API:** from `php artisan route:list --json`.

Regenerate them whenever the schema or routes change so the documentation never goes stale.

Limits of this documentation (stated plainly)

- Prepared by static code review, without penetration or load testing.
- Not every controller was read line by line; section 7 of [08](#) lists what was not reviewed.
- The Flutter code (~73K lines) was reviewed at the structure and services level, not screen by screen.
- Some Arabic-only details (UI strings, code comments) are intentionally left untranslated.

Product Overview

What is Edu-Bridge?

Edu-Bridge is an integrated academic management system for an educational institute. It connects **six roles** across several channels:

- **Mobile/web app (Flutter)** for all roles.
- **Web dashboards (Laravel Blade)**, one per role, each with its own login page.
- **Telegram bot** for students.

Goal: automate the daily academic cycle (timetables, attendance, assignments, grades, administrative requests) and connect the student with their parent, teachers and administration in one place.

Roles

Role	role_id	Main capabilities
Admin	1	Manage accounts, departments, programs and courses; assign heads of department; reports; announcements; bulk notifications; activity log; theme customization
Teacher	2	Lectures and materials, QR attendance sessions, assignments and grading, grades, quizzes, performance reports, parent summons. Advisors have extra tools (cohort attendance export, daily summary)
Student	3	Timetable and exams, attendance (QR + face + location), assignments and lectures, grades and academic card, leave and service requests, chat, AI assistant
Parent	4	Follow children (attendance, grades, assignments), approve leave permissions, request reports and meetings, answer summons
Head of department	5	Timetables and exams, approve leave and service requests, request behavioral/grade reports from teachers, create accounts, announcements, summons
Affairs officer	6	Pre-registered university IDs, account approval, semesters and promotion, academic card and transcripts, course weights and cohort reports, device unlock, student-parent linking, activities and calendar

Main features

Smart attendance

The teacher opens a session and a **QR code that rotates every 30 seconds** is generated (the session lasts 10 minutes). The student scans it in the app and the server checks: 1. QR validity and scan time 2. Student eligibility for the course (year / specialization / enrollment) 3. No duplicate record 4. **Device binding** (one device per student) 5. **Geolocation** within the classroom radius 6. **Face match** (MobileFaceNet model on-device, 70% threshold)

There is also **offline attendance**: the app stores the scan time and syncs later according to the department policy (`anytime` or `same_day`).

Automatic absence warnings

Absence days	Action
7	First warning to the student
10	Second warning + automatic parent summons
15	Final warning and referral to administration and the head of department

Communication

- **Real-time chat** (Pusher) with attachments, voice notes, groups, edit, delete, forward and disappearing messages, governed by a **permission matrix** that defines who can message whom.
- **Notifications** in-app + push (FCM) + Telegram.
- **Announcements and activities** with a target audience and images.

Administrative request cycle

Leave: student -> parent -> head of department -> Affairs. Student services (appeal, documents, make-up exam, device unlock): student -> head of department -> Affairs/Admin.

Reports and export

Academic card (PDF/Excel), study and exam timetables (PDF/Excel/image), attendance reports (PDF/Excel), cohort reports, performance reports and advisor reports.

AI assistant

In-app chat powered by **Google Gemini** with live context from the user's data, and a local fallback engine if no key is configured or the call fails.

Telegram bot for students

Timetable, grades, attendance, lectures, submit excuses and leave, QR scanner, and instant notifications for grades and absences. Also used to send OTP codes and recover passwords.

Security and sessions (summary)

- One active session per channel (web / mobile).
- Account lockout for 15 minutes after 5 failed logins, and per-IP rate limiting.
- A student is bound to one device; unlocking is requested through Affairs.
- An activity (audit) log for important actions.
- Details in [07-security.md](#) and [08-project-status.md](#).

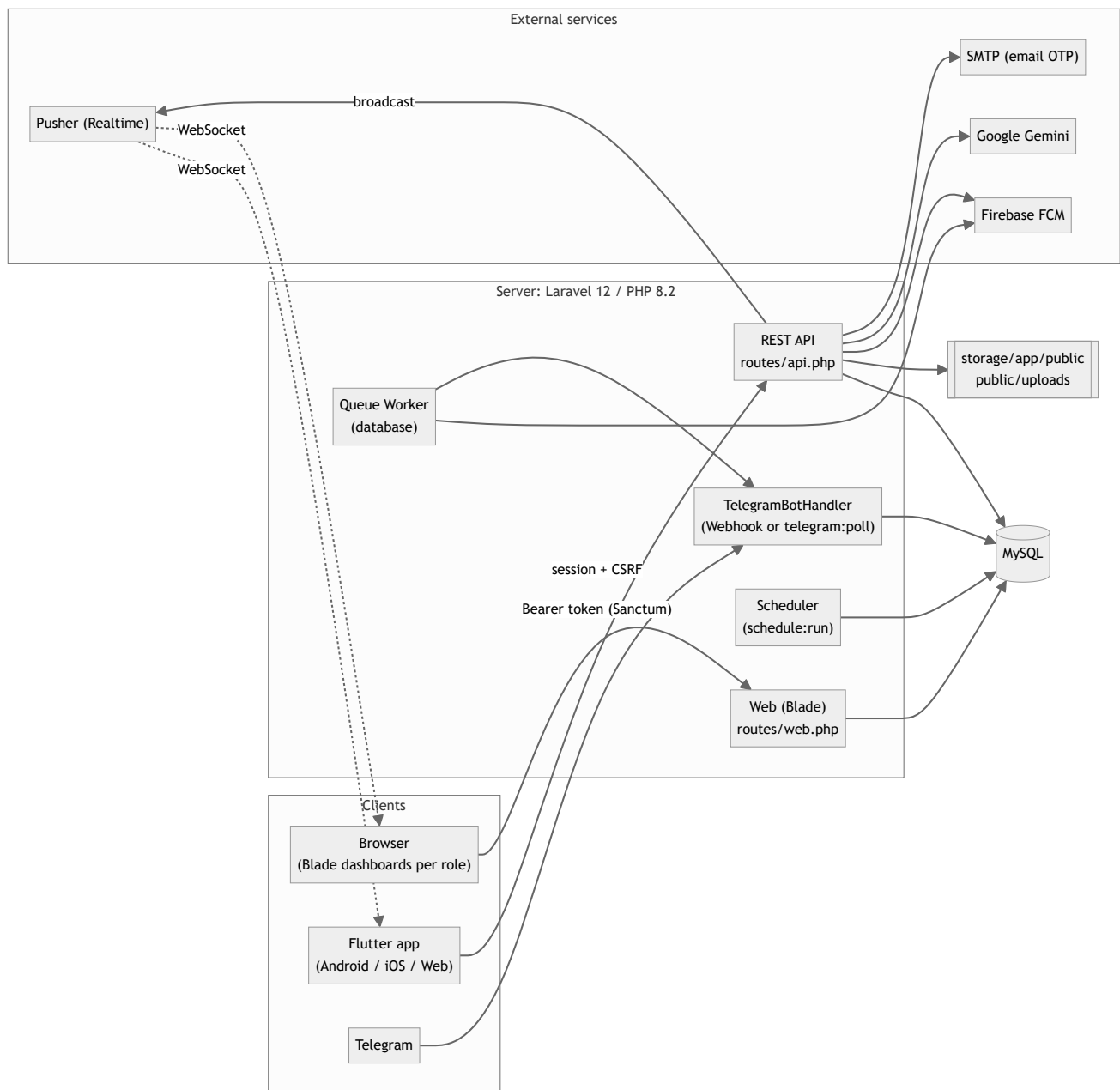
Project size

Component	Approximate size
Backend (PHP)	~36K lines, 48 controllers
Web UI	142 Blade pages
Flutter app	~73K lines, ~170 files
Database	65 tables + 3 views (about 120 migrations)

Component	Approximate size
API	330 endpoints
Web routes	~370 routes

Architecture

Overview diagram



Backend layers

Layer	Location	Role
Routes	<code>routes/api.php</code> , <code>routes/web.php</code> , <code>routes/channels.php</code> , <code>routes/console.php</code>	Define the interfaces and attach middleware (role, single session, throttle)
Middlewares	<code>app/Http/Middleware/</code>	<code>RoleMiddleware</code> (API), <code>Check*Role</code> (web), <code>EnsureSingleApiSession</code> , <code>EnsureSingleWebSession</code>

Layer	Location	Role
Contr ollers	app/Http/Controllers/{Api,Web,WebHead}	Request logic. Very large and mix business logic with presentation (see section 5 of project status)
Servic es	app/Services/	FcmService, TelegramService, TelegramBotHandler (+ role traits in app/Services/TelegramBot/), AbsenceWarningService, StudentAcademicService, PDF/Excel/timetable-image services
Obser vers	app/Observers/	AttendanceObserver (absence warnings + Telegram notice), GradeObserver (grade notice)
Supp ort	app/Support/	SingleSessionGuard, LoginThrottleGuard
Traits	app/Traits/	FaceRecognitionTrait, HandlesMessagesTrait, NormalizesAccountCredentialsTrait
Jobs/ Event s	app/Jobs, app/Events	Background FCM/Telegram sending, message broadcast (MessageSent)
Mode ls	app/Models/	42 Eloquent models with custom primary keys
Com mand s	app/Console/Commands/	Maintenance and generation commands (below)

Two interfaces over the same logic

	API (for the app)	Web (Blade)
Authenticat ion	Sanctum Bearer token	Laravel session + CSRF
Role protection	role:student ...	student / teacher / hod / affairs / admin / parent
Single session	Current token (current_token_id)	current_session_id + 20-minute idle window
Login	POST /api/login	One unified page for every role: /login (handled by UnifiedAuthController; the per-role login URLs were removed)

Core logic is duplicated between the two interfaces (e.g. QR attendance has an API copy, a Web copy and a Telegram copy). This is the project's biggest technical debt.

Real time and notifications

- **Chat:** the MessageSent event is broadcast through Pusher on **private** channels chat.{user_id} (receiver and sender); the app listens to them, **with a 2-second polling fallback** in parallel.
- **Notifications:** stored in the notifications table; FcmService sends push (HTTP v1 with a service account); the app reads them by polling every 30 seconds; students also get Telegram messages.
- **Channel authorization:** POST /api/broadcasting/auth.

Scheduled tasks and commands

Command	Schedule	Purpose
<code>attendance:daily-summary</code>	Daily 22:00	Today's attendance summary for each course advisor
<code>logs:clean --days=90</code>	Daily	Purge old activity log entries
(scheduled closure in <code>bootstrap/app.php</code>)	Every 5 min	Delete expired disappearing messages and their attachments
<code>telegram:poll</code>	Manual / service	Run the bot by polling instead of a webhook
<code>schedules:generate [--fresh]</code>	Manual	Generate sample timetables and exams
<code>grades:generate [--fresh]</code>	Manual	Generate sample grades
<code>db:export</code>	Manual	Export the database

This requires a **cron** entry running `php artisan schedule:run` every minute, and a **queue worker** `php artisan queue:work`.

Flutter app

Aspect	Detail
Entry point	<code>lib/main.dart</code> : loads settings (theme, font, language), initializes <code>ApiService</code> and Firebase in the background, then <code>_AppRouter</code> reads <code>token</code> and <code>role</code> from <code>SharedPreferences</code> and routes to the role's home screen
State	<code>provider</code> (<code>ChatService</code>) + <code>ValueNotifier</code> for settings + <code>SharedPreferences</code>
Networking	<code>dio</code> through <code>ApiService</code> (+ <code>SingleSessionInterceptor</code> handling <code>LOGGED_IN_ELSEWHERE</code>), with service layers: <code>student_services</code> , <code>admin_services</code> , <code>affairs_services</code> , <code>parent_services</code>
Structure	<code>lib/screens/<role>/</code> (admin, Affairs_Officer, Head of department, parents, student, teacher, shared, auth, onboarding) and <code>lib/widgets/</code> for shared components
Face	<code>google_mlkit_face_detection</code> for detection + <code>tflite_flutter</code> with <code>mobile_face_net.tflite</code> (112x112 -> 192 dimensions)
Server discovery	Automatic: USB (<code>127.0.0.1</code> via ADB reverse), then known LAN addresses, then subnet scan
Languages	Arabic/English, dark mode, 3 font sizes, and a primary color the admin controls (<code>/api/system/settings</code>)

Current runtime environment notes

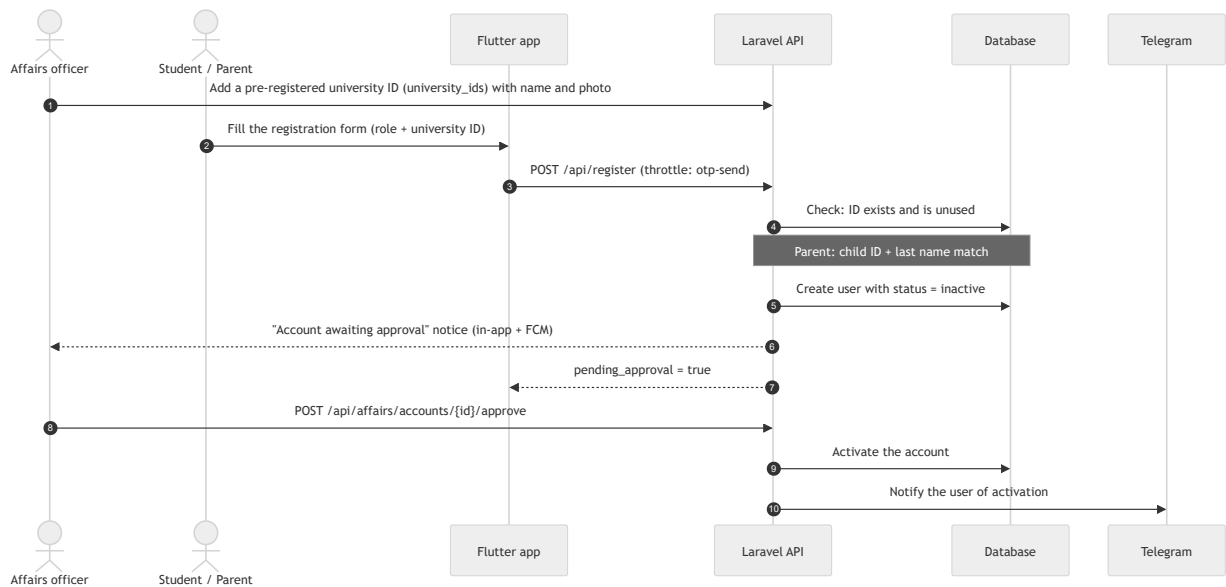
- The current setup targets **local laptop operation**: `php artisan serve` (port 8000), a USB tunnel, and an `ngrok` gateway (`ngrok.exe` has been removed from the repository).
- There is no Docker, no CI/CD, and no configured production web server. Details in [06-setup-and-deploy.md](#).

Key Flows

Diagrams use Mermaid (rendered automatically on GitHub and the documentation site). Every flow is based on the real code; the source file is named.

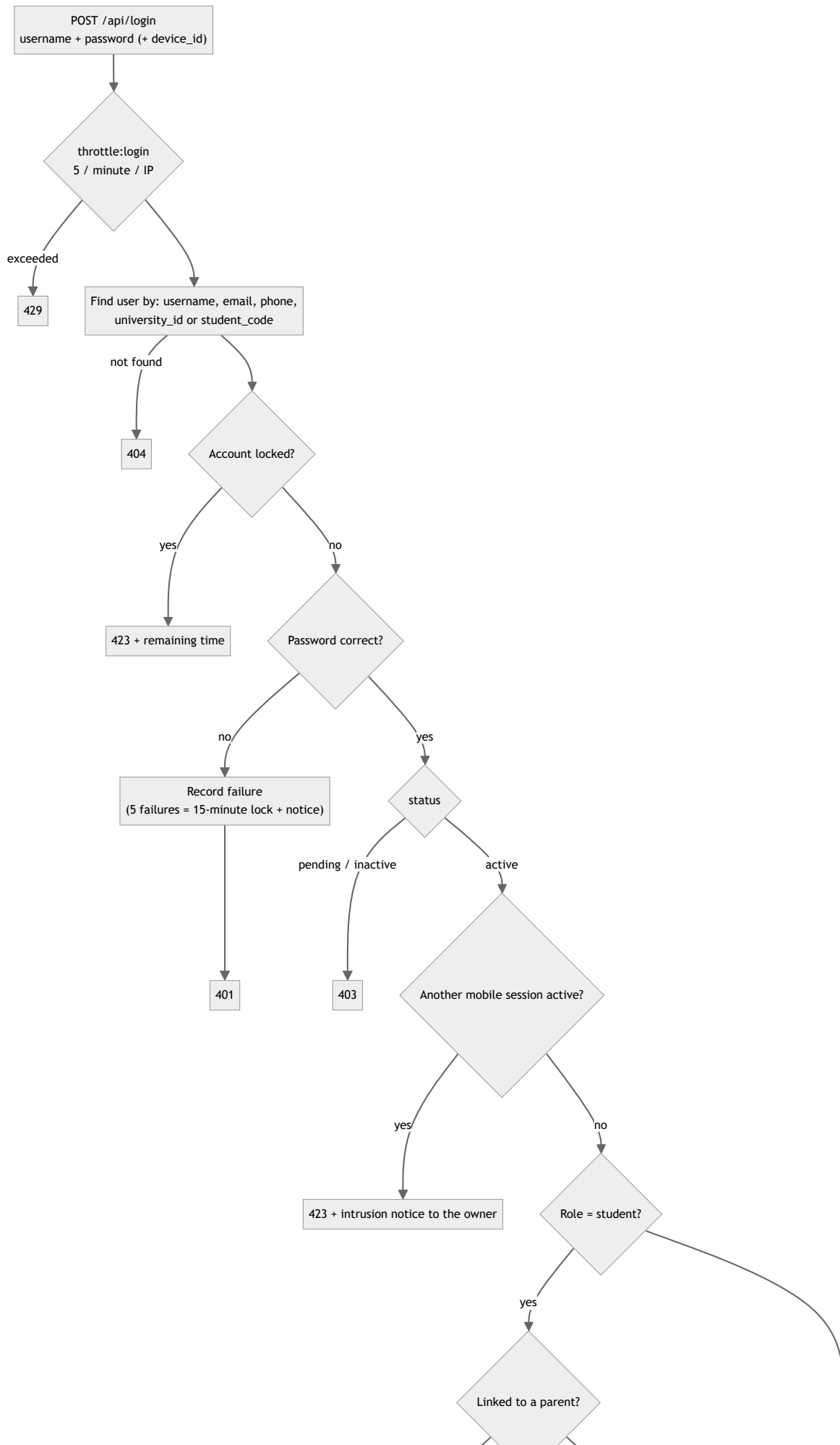
1) Account creation and activation

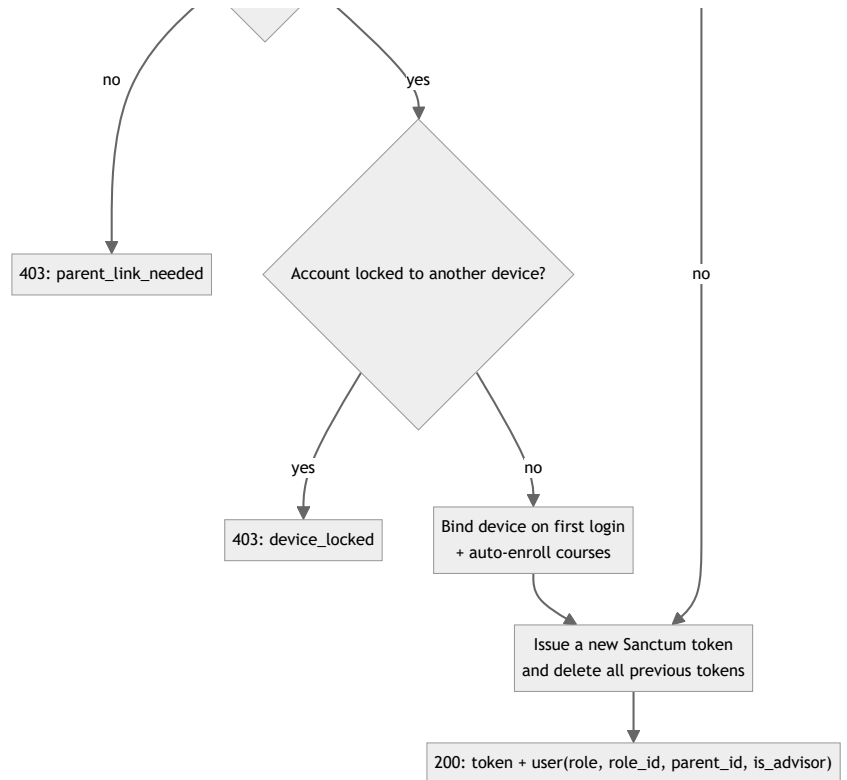
Self-registered student and parent accounts go through Affairs review. (Source: `Api\AuthController::register`, `AffairsController::approveAccount`)



2) Login (API)

(Source: `Api\AuthController::login`, `SingleSessionGuard`, `LoginThrottleGuard`)

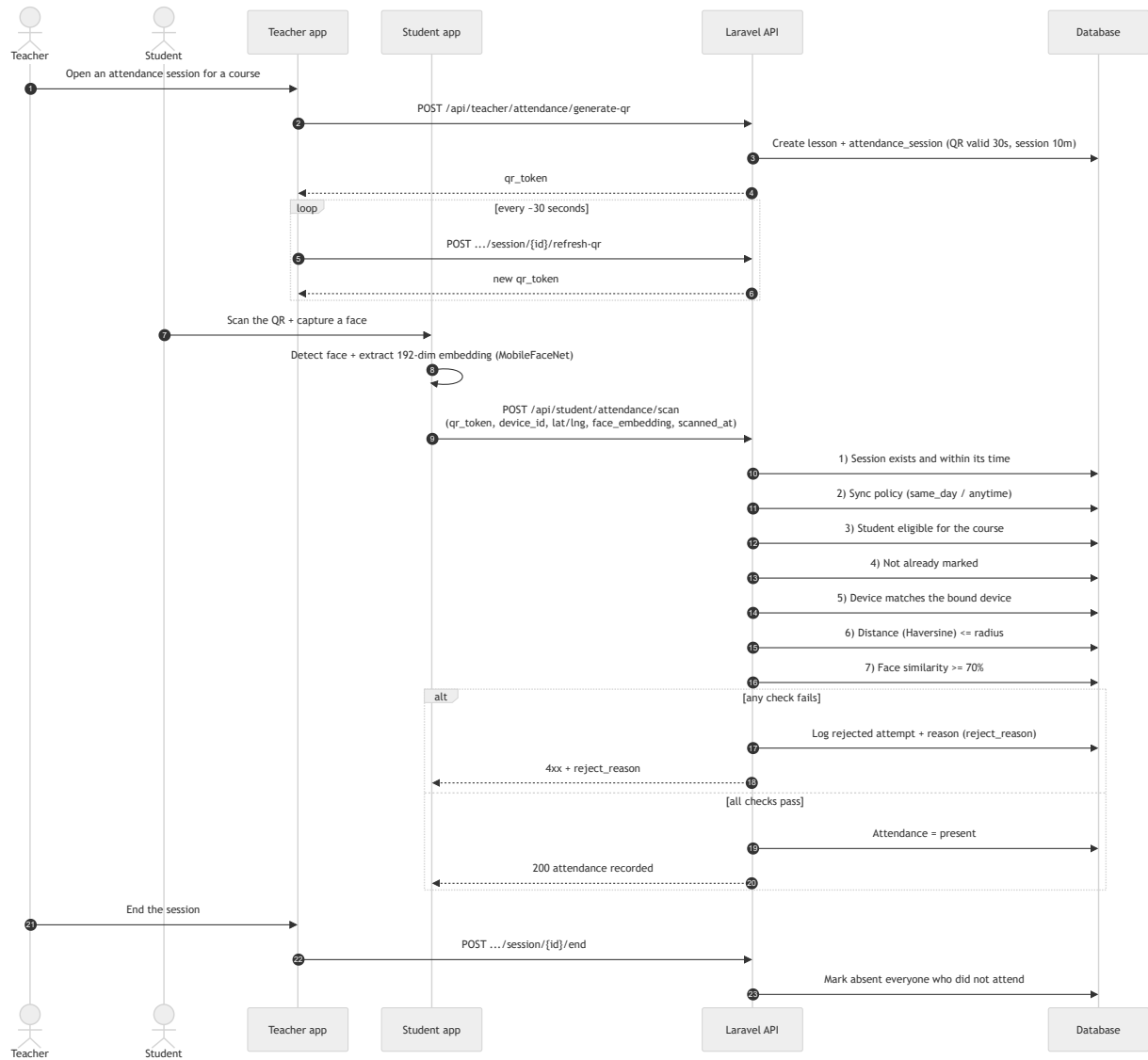




Alternatives: **email OTP login** (/login-otp/send then /login-otp/verify); the web has one login page per role, plus **face verification** for students signing in from multiple devices.

3) Attendance (QR + device + location + face)

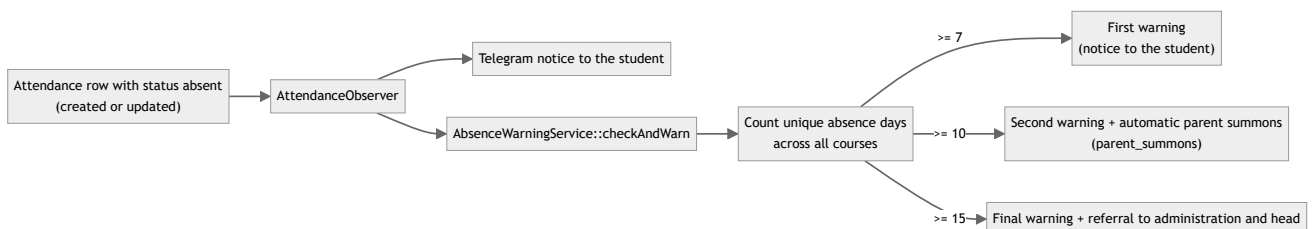
(Source: Api\TeacherController::generateQrSession and Api\StudentController::scanAttendanceQr)



Reject reasons (`reject_reason`): `expired_qr`, `sync_timeout`, `lesson_not_found`, `device_mismatch`, `location_too_far`, `face_mismatch`, plus eligibility and duplicate reasons.

4) Automatic absence warnings

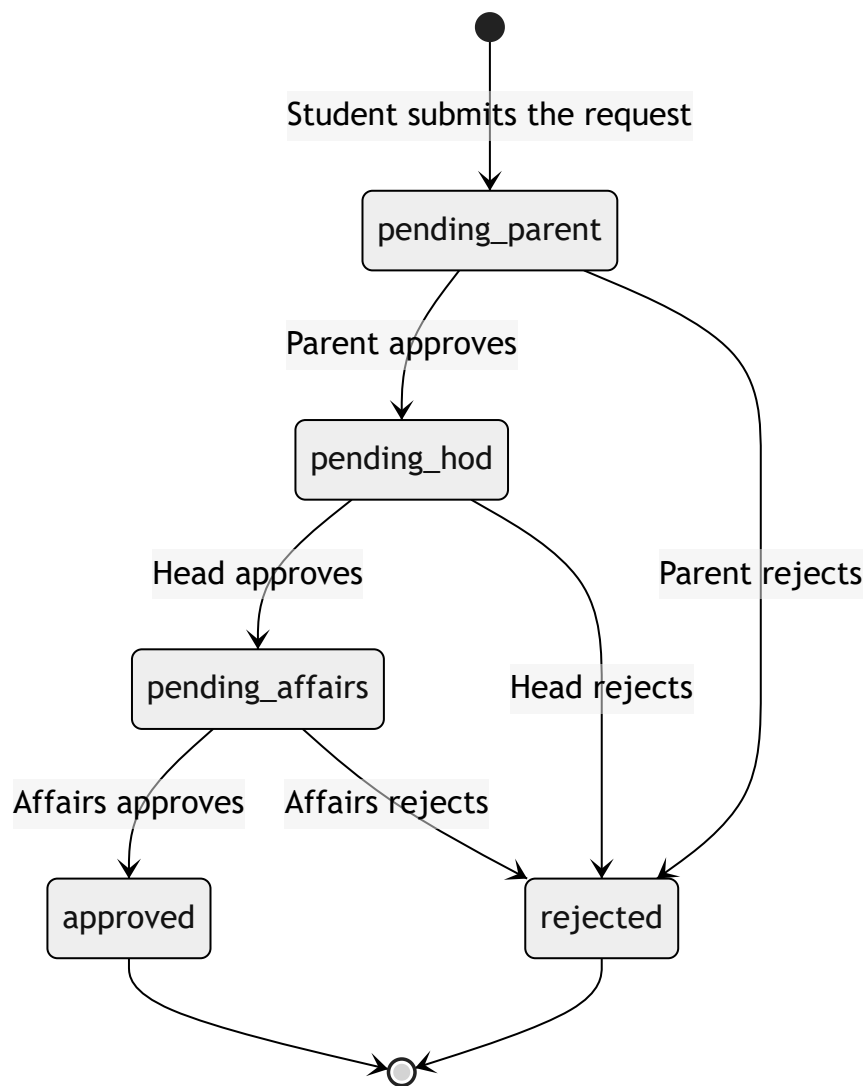
(Source: `AttendanceObserver`, `Services\AbsenceWarningService`)



Each level is issued **once** per student (`student_warnings`). See note B-07 in the project status.

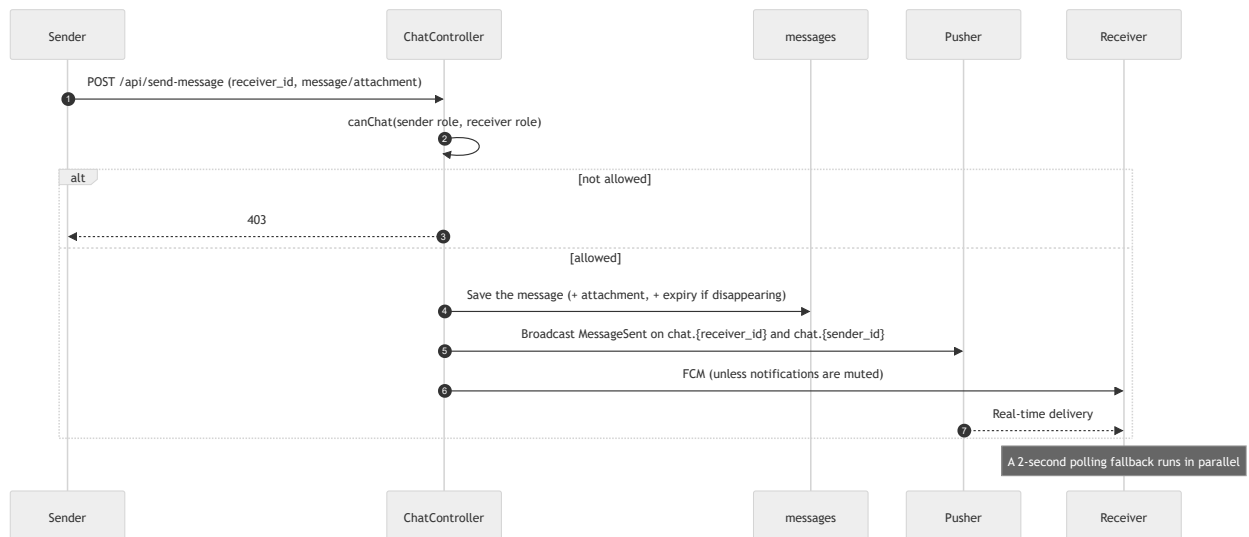
5) Leave request

(Source: StudentController::requestAbsence -> StudentParentController::respondLeaveRequest -> Api\DepartmentHeadController::respondLeaveRequest -> AffairsController::updateLeaveStatus)



All leave requests live in `leave_requests` and run through one workflow (`LeaveWorkflow`) for the web, the Flutter app and the bot. The old `absence_requests` table was copied into it and dropped.

6) Chat



Who can message whom:

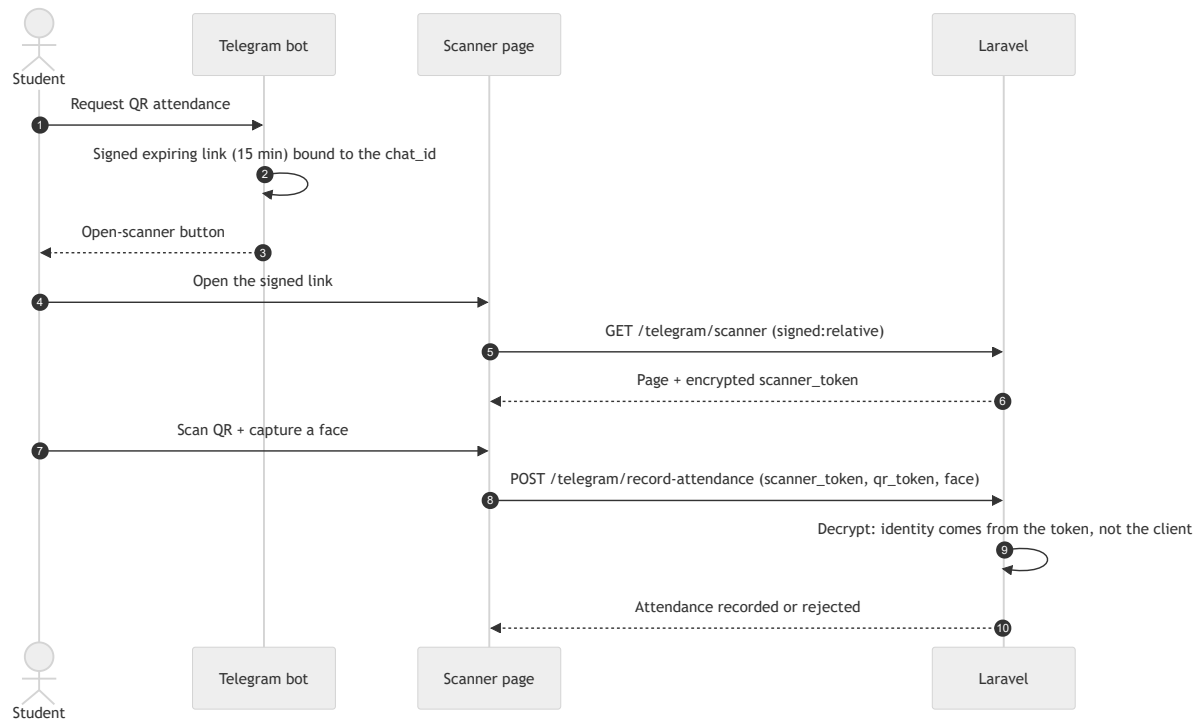
Sender	Can message
Student	Head of department, teacher, admin
Teacher	Teachers, students, head of department
Parent	Admin, head of department
Head of department	Parent, teacher, student, admin
Admin	Head of department, Affairs, teacher, student
Affairs	Admin

7) Password recovery

- **API:** forgot-password sends an OTP, then reset-password (email + code + new password).
- **Web:** through Telegram: send-otp -> verify-otp (5 attempts maximum) -> reset ; the code is kept in the browser session and is valid for 15 minutes.

8) Telegram scanner

(Source: TelegramBotHandler::handleQrAttendanceMenu, TelegramWebhookController)



Statuses: `verified` for a face match of 70% or more, `first_time` for the first embedding, `suspicious` when face data is missing or the match is weak (the teacher is alerted). Face can be made mandatory with `ATTENDANCE_REQUIRE_FACE=true`.

Setup and Deployment

Honesty note: this guide is based on reading the configuration files and scripts that exist. Local-run steps are partially verified (the migrations were run from scratch successfully). **Production deployment** is a set of recommendations, because the repository contains no configured production deployment.

1) Requirements

Component	Version
PHP	^8.2 (with <code>pdo_mysql</code> , <code>gd</code> , <code>mbstring</code> , <code>openssl</code> , <code>zip</code> , <code>fileinfo</code>)
Composer	2.x
MySQL	8.x (tested 8.4) with <code>utf8mb4</code>
Node.js	18+ (for Vite and <code>puppeteer-core</code>)
Flutter	SDK with Dart ^3.10
External accounts	Firebase (FCM), Pusher, Telegram Bot, Google Gemini (optional), SMTP

2) Running the backend locally

```
# 1. Dependencies
composer install
npm install

# 2. Configuration
cp .env.example .env          # then edit the values (section 3)
php artisan key:generate

# 3. Database
# create an empty database (utf8mb4_unicode_ci), then:
php artisan migrate --force
php artisan db:seed           # sample data (section 4)

# 4. Run
php artisan serve --host=0.0.0.0 --port=8000
npm run dev                  # or: npm run build for production
php artisan queue:work       # queue worker (FCM / Telegram)
php artisan schedule:work    # scheduled tasks locally (instead of cron)
```

Or the ready shortcut: `composer run dev` (runs the server, queue, logs and Vite together).

.env.example is incomplete: `TELEGRAM_WEBHOOK_SECRET` and `GEMINI_API_KEY` were added to it. The remaining standard Laravel variables (database, mail, Pusher) must be copied from section 3.

3) Environment variables (`.env`)

Variable	Purpose	Note
APP_ENV	local for development, production for production	WARNING: in local a fixed OTP 123456 is used (see security)
APP_DEBUG	false in production	
APP_URL	Server URL	Used to build file links
DB_CONNECTION/HOST/PORT/DATABASE/ USERNAME/PASSWORD	MySQL	
SESSION_DRIVER	file currently	database or redis for production
QUEUE_CONNECTION	database	Requires queue:work
BROADCAST_CONNECTION	pusher	
PUSHER_APP_ID/KEY/SECRET/CLUSTER	Real-time chat	The app key and cluster (eu) are written in chat_service.dart and must match
MAIL_MAILER/HOST/PORT/USERNAME/PASSWORD/ FROM_ADDRESS	OTP emails	WARNING: the current value log sends nothing
TELEGRAM_BOT_TOKEN	The bot	From BotFather
TELEGRAM_WEBHOOK_SECRET	Secret to verify the webhook	Required in production
TELEGRAM_FORWARD_NOTIFICATIONS	Copy every app notification to the user's Telegram	true (default); false to turn off
GEMINI_API_KEY	AI assistant	Optional; without it the local engine is used
SESSION_LIFETIME	Minutes (default 20)	

A file outside .env : **storage/app/firebase-service-account.json** (Firebase service account for sending FCM). It is intentionally not in the repository.

4) Sample data

php artisan db:seed runs DatabaseSeeder . There are many seeders under database/seeder/ (departments, programs, courses, teachers, students, parents, attendance, grades). Extra generation commands:

```
php artisan schedules:generate --fresh # timetables and exams
php artisan grades:generate --fresh # sample grades
```

WARNING: check the passwords of the seeded accounts and change or remove them before any deployment.

5) Running the Telegram bot

Two options: 1. **Webhook (production):** needs a public HTTPS URL, and the webhook is registered with Telegram using a secret_token equal to TELEGRAM_WEBHOOK_SECRET :

`https://api.telegram.org/bot<TOKEN>/setWebhook?url=`

`<APP_URL>/api/telegram/webhook&secret_token=<SECRET>` 2. **Polling (development / no public URL):**

php artisan telegram:poll (or start_bot.bat on Windows).

6) Running the Flutter app

```
cd Edu_Pridge_flutter
flutter pub get
flutter run                # Android / emulator
flutter run -d chrome      # web
```

Server address: currently written in `lib/services/api_service.dart`. The app automatically tries: `127.0.0.1` (USB via `adb reverse tcp:8000 tcp:8000`), then fixed LAN addresses, then a subnet scan. For web it uses `http://127.0.0.1:8000`. **It should be replaced by environment configuration (flavors / `--dart-define`)** before any deployment.

Port note: the script `سيرفر/سيرفر.bat` runs the server on **8000** (primary, what the app assumes) and a second copy on **8001** (fallback for teammates who can only use that port; a separate scheduled task `EduBridge Server 8001`). The app's discovery tries 8000 first, then 8001. The ngrok tunnel and the Telegram bot use 8000 only.

Firebase: `android/app/google-services.json` exists. For web, the Firebase keys are written in `lib/main.dart`.

Face model: `assets/models/mobile_face_net.tflite`. WARNING: this file's license is not explicitly documented by its source (see `assets/models/NOTICE.md`), and must be settled before any commercial use.

7) Production deployment recommendations

The repository is not set up for it, so these are guidelines:

1. **Server:** Linux + Nginx + PHP-FPM + MySQL. Web root `public/`.
2. **.env:** `APP_ENV=production, APP_DEBUG=false, real SMTP, SESSION_DRIVER=database|redis, SESSION_SECURE_COOKIE=true`.
3. **HTTPS is mandatory** (the code forces it in `production`).
4. **Cron:** `* * * * * cd /path && php artisan schedule:run >> /dev/null 2>&1`
5. **Supervisor** for the queue worker: `php artisan queue:work --tries=3`.
6. **Storage:** `php artisan storage:link`; block PHP execution inside `storage/` and `public/uploads/` in the Nginx config; move `public/uploads/faces` outside the public root.
7. **Optimization:** `php artisan config:cache && route:cache && view:cache`, and `composer install --no-dev --optimize-autoloader`.
8. **Backup:** `php artisan db:export` or a scheduled `mysqldump`.
9. **Flutter:** build `flutter build apk --release` (Android) with the production address, and sign the app.

8) Common problems

Symptom	Cause and fix
Chat is not real time	Check <code>BROADCAST_CONNECTION=pusher</code> , the Pusher keys, and that <code>/api/broadcasting/auth</code> returns 200. (A 2-second polling fallback runs)
OTP emails do not arrive	<code>MAIL_MAILER=log</code> (writes to <code>storage/logs</code> only)
FCM notifications do not arrive	Missing <code>storage/app/firebase-service-account.json</code> , or no <code>queue:work</code>
Disappearing messages are not deleted	No cron for <code>schedule:run</code>
The app cannot find the server	<code>adb reverse tcp:8000 tcp:8000</code> , or set the address in <code>api_service.dart</code>
429 error	Rate limit exceeded (login 5/min, OTP 10/hour per email)
423 on login	The account is locked (5 failures) or signed in on another device

9) Distributing and updating the Android app

The app is not on Google Play; it is downloaded **directly from the institute server** through a public page:

- Download page: `/app` (e.g. `http://82.137.250.43:8080/edu_bridge/public/app`) with the download button, install steps, a QR code and share buttons.
- Direct download: `/app/download` (64-bit build) and `/app/download?abi=v7a` (32-bit build for old phones).
- Version check used by the app: `GET /api/app-version[?abi=arm64|v7a]` (public, no login).
- Files live in `storage/app/app-release/`: `edubridge.apk`, `edubridge-v7a.apk` and `release.json`.

Publishing a new version:

1. Raise the number after `+` in `pubspec.yaml` (e.g. `1.0.1+2`).
2. From the Flutter app folder: `flutter build apk --release --split-per-abi`.
3. Copy `app-arm64-v8a-release.apk` to `edubridge.apk` and `app-armeabi-v7a-release.apk` to `edubridge-v7a.apk`.
4. Edit `release.json` (`version_name`, `version_code`, `changelog`; change `min_version_code` only to force an update).
5. Upload the backend folder to the server.

Signing rule: an update installs over the existing app (keeping its data) only if the `applicationId` and the signing key are the same and the version number is higher. The release key (`android/edubridge-release.jks` and `android/key.properties`) is not in Git and must be backed up. Full details are in `APP_RELEASE_GUIDE_FOR_AI.md` at the project root.

Security Design

This file describes the **existing controls**. For open and fixed vulnerabilities see [08-project-status.md](#).

1) Authentication

Channel	Mechanism	Details
App (API)	Laravel Sanctum, Bearer token	One current token per user (<code>users.current_token_id</code>); issuing a new token deletes the previous one. Tokens expire after 30 days (<code>SANCTUM_EXPIRATION</code>), and the app returns to login on 401
Web	Laravel session + CSRF	One active session (<code>current_session_id</code>) with a 20-minute idle window; afterwards a new login is allowed
Login alternative	Email OTP (<code>login-otp</code>)	6-digit code via <code>random_int</code> . A fixed dev-only code applies only when <code>OTP_FIXED_CODE</code> is set explicitly in a local/testing environment
Student on multi-device web	Face verification	<code>/student/face-verification</code>

Passwords are hashed with `bcrypt` (`Hash`). Minimum length is 8 characters when set by the user (6 for staff-created accounts for now).

2) Authorization

- **API:** `RoleMiddleware` with the roles allowed per route group (`role:student` , `role:teacher` , `role:head` , `role:admin` , `role:parent` , `role:affairs,admin`).
- **Web:** one middleware per role (`CheckStudentRole` ...).
- **Ownership (record level):** every controller must verify the resource belongs to the user. Applied explicitly in: child details for parents (`ParentController`), chat attachment download, teacher attendance sessions, and student/parent data (after the fixes). **This is the largest source of vulnerabilities in the project; every new endpoint that takes an id must check it.**
- **Chat:** the `canChat` matrix defines who can message whom, and the private channels `chat.{id}` are authorized by matching `user_id` .

2.1) Self-registration

Self-registration is limited to the `student` and `parent` roles. No other role can be chosen. Every new account needs **Affairs approval** and a pre-registered university ID.

3) Brute-force and abuse protection

Control	Value	Where
Login limit	5 / minute / IP (<code>throttle:login</code>)	<code>AppServiceProvider</code>
Account lockout	After 5 consecutive failures, 15 minutes, with a notice to the owner	<code>LoginThrottleGuard</code>

Control	Value	Where
OTP sending	3 / minute (login); 5 / minute + 10 / hour per email (register / forgot password)	login-otp, otp-send
OTP verification	10 / minute / IP + 10 / hour per email; web: 5 attempts then the code is invalidated	otp-verify
General API	240 requests / minute per user	api
Single session	Refuse login from a second device + notice to the owner	SingleSessionGuard

4) Attendance integrity

The six layers (rotating QR, bound device, location, face, eligibility, no duplicates) are described in [05-key-flows.md](#). **Their limits:** device, location, time and face embedding are all values sent by the client (a mobile app that can be modified), and there is no server-side liveness check. They are therefore **deterrent controls**, not an absolute guarantee. Details in items S-10 to S-12.

5) Sensitive data

Data	Where stored	Note
Passwords	users.password (bcrypt)	
Reference face embedding	students.face_embedding (JSON, 192 numbers) + students.reference_photo	Biometric data; requires consent and a retention policy
Attendance photos	storage/app/private/faces/ (private)	Served through GET /api/attendance/{id}/face to authorized users only; faces:secure moves old ones
Service keys	.env and storage/app/firebase-service-account.json	Outside git
Activity log	user_activities	Entries older than 90 days are purged

6) Transport

- HTTPS is forced in production (URL::forceScheme).
- CORS is restricted: CORS_ALLOWED_ORIGINS + localhost for development (it was open *).
- CSRF protection is enabled for web **with no exceptions**. Sessions are encrypted by default.

7) Audit and monitoring

- user_activities: logins, failures, rejections, lockouts and admin operations, via UserActivity::log.
- attendance: rejected attendance is recorded with reject_reason.
- Laravel logs in storage/logs. There is no external monitoring or alerting.

8) Guidance for the next developer

1. Any new endpoint that takes {id} must verify resource ownership before any read or write.
2. Do not use rand() for codes; use random_int().
3. Any file upload: a mimes: allow-list, and the extension taken from the content, not the file name.
4. Never put keys in code or the repository.
5. Add a Feature test for every new route verifying that another role gets 403.

9) Telegram bot and password reset hardening (2026-10-07)

The bot and the password reset were reviewed after the AR/EN and bot suite merge. Fixed, each with feature tests that fail on the old code:

Area	Problem	Now
Pass word reset (send Reset Otp)	For an account with no linked Telegram, the id typed in the request was saved on the account and the OTP was sent to it (account takeover). The OTP was also returned in the JSON when delivery failed.	The OTP goes only to the chat already linked to the account (linked from the bot with the password) and is never returned. Unknown / inactive / not-linked / failed delivery return the same 422 (no account enumeration).
Bot sign-up	Any university id not already used was accepted; a parent only needed a student's university id.	The id must exist in <code>university_ids</code> and be unused; the parent's family name must match the student's (same rule as the app). Passwords: 8 characters.
Bot sign-in	No brute-force protection.	<code>LoginThrottleGuard</code> (5 failures = 15 min lock, shared with the web login).
Bot buttons	Role-gated menus only; the action handlers never checked the caller.	Central role guard by button prefix (<code>admin_</code> , <code>affairs_</code> , <code>hod_</code> , <code>teacher_</code> , <code>parent_</code>) and ownership checks (<code>CallbackAuthorization</code>): parent to own children, student to own absences and enrolled courses, teacher to own courses/sessions/submissions, head to his department's requests.
Admin lectures	<code>content_url</code> (free text written by the teacher) could point to any server file, and a hard-coded CV was served for missing files.	Files must resolve (realpath) under <code>storage/app/public</code> , <code>public/storage</code> or <code>public/uploads</code> ; no fallback file.
Semesters / promotion	<code>semester_name</code> column did not exist (SQL error); the two promotion routes behaved differently.	One shared promotion routine (level, academic year, auto-enrolment, one transaction per student, activity log).
Head of department (bot and web)	Lists, details and decisions fell back to "show everything" or never checked the department; leaves were approved without Student Affairs; notifications and announcements reached the whole university; LIKE matching.	Everything goes through <code>Access</code> (exact department, workflow stage <code>pending_hod</code> , fail closed without a department). Leave approval moves to <code>pending_affairs</code> . Audience = the head's department (<code>Access::headAudienceUserIds</code>).
Affairs / Admin (bot)	Decisions had no stage check; admin "reject account" deleted any user (even an active one). Affairs leave buttons never worked (id parsed as 0).	Affairs decides in <code>pending_affairs</code> , admin in <code>pending_admin</code> , account reject/approve only on pending accounts; button parsing fixed.

Area	Problem	Now
Bot routing and crashes	Two head buttons (courses, announcement) went to the dashboard; appointments, affairs student search and lecture-file details crashed on wrong columns / tables / model key.	Fixed; a test presses every button of the six keyboards and a crawler presses every inline button on realistic data.
Student leave scenario (all channels)	The web stored requests in the legacy <code>absence_requests</code> table, the Flutter app and the bot in different ways; the web parent form inserted every request twice; parent/affairs steps had no stage check; next-stage notices went to every head.	One service, <code>LeaveWorkflow</code> , for the web, the Flutter API and the bot, on <code>leave_requests</code> : student, parent (<code>pending_parent</code>), head of the student's department (<code>pending_hod</code>), student affairs (<code>pending_affairs</code>), approved. Each step only in its stage; notices to the student's department head only; final decision to student, parents, head and advisor. The legacy <code>absence_requests</code> rows are copied to <code>leave_requests</code> by the migration <code>2026_10_08_100000</code> , and the table is then dropped by <code>2026_10_08_110000</code> (it refuses to drop while a row is uncopied); the unused legacy endpoints <code>parent/permissions</code> and <code>teacher/absence-requests</code> were removed. Run <code>php artisan migrate</code> on the server. The bot gained "my leaves" and the parent's "children's leaves" approval.
Notifications	Every app notification is copied to Telegram.	Kept, but <code>TELEGRAM_FORWARD_NOTIFICATIONS=false</code> turns it off.

Still open (not fixable by code):

1. A Telegram bot token committed in June 2026 and removed in September is still in the Git history of a **public** repository. Revoke it with BotFather (`/revoke`) and keep the new token only in `.env`.
2. The institute server is served over plain HTTP.
3. `TelegramBotHandler` was split into role traits under `app/Services/TelegramBot/`; the bot still has little test coverage outside sign-up, sign-in, role and ownership checks.

Current Project Status

Review date: 2026-10-05 **Scope:** the Laravel repository (edu_bridge_backend), with references to the Flutter app (Edu_Pridge_flutter) **Method:** manual code review (static review). **No dynamic penetration tests were run and the project was not load tested.** Every item is marked:  confirmed by reading the code, or  possible and needs verification.

1. Executive summary

Dimension	Assessment
Functional completeness	High. 6 roles, a mobile app and full web dashboards, real-time chat, QR + face + location attendance, automatic absence warnings, a Telegram bot, an AI assistant
Security maturity	Good. All known critical, high and medium issues are closed, with 104+ automated tests . Documented accepted risks remain (S-11, S-15) plus production settings
Code quality	Medium. Large controllers and web/API duplication remain, but there is now a test safety net for authorization and attendance
Production readiness	Ready after environment setup (section 6) and running <code>php artisan migrate</code>
Documentation before this set	Almost none (default Laravel/Flutter READMEs)

2. What was fixed in this session

#	Fix	File
1	Removed <code>/create-student</code> (created a student account with password <code>123456</code> , with no protection)	<code>routes/web.php</code>
2	Removed <code>/dev/reset-schedules</code> (ran <code>truncate</code> on <code>schedules</code> , open to any logged-in user)	<code>routes/api.php</code>
3	Removed <code>/user/profile/{id}</code> and <code>/parent/notifications/{id}</code> (read any user's data/notifications with no ownership check)	<code>routes/api.php</code>
4	Removed the public <code>/parent/info/{user_id}</code> (exposed name and phone with no token) and a duplicate <code>/system/settings</code>	<code>routes/api.php</code>
5	<code>/ai/chat</code> now requires authentication (protects the Gemini key from public use)	<code>routes/api.php</code>
6	<code>/student/info/{id}</code> : added an ownership check (a student sees only themselves; a parent only their linked children)	<code>routes/api.php</code>
7	<code>/file/{path}</code> : stopped serving a random substitute PDF, and restricted access with <code>realpath</code> inside <code>storage/app/public</code>	<code>routes/api.php</code>
8	<code>/storage/{path}</code> : path traversal protection via <code>realpath</code>	<code>routes/web.php</code>
9	Untracked <code>ngrok.exe</code> , <code>*.sql</code> files and the SQLite file from git, and updated <code>.gitignore</code>	<code>.gitignore</code>

Second batch (high-severity fixes)

#	Fix	File
10	S-01/S-02: a parent is limited to their linked children (<code>parentOwnsStudent</code>) in: performance, assignments, permissions, answering an absence permission, answering a leave request	<code>StudentParentController.php</code>
11	S-05: a teacher reaches only their own attendance sessions (<code>findOwnedSession</code>): end session, attendee list, QR refresh. Face reset is limited to students of their courses	<code>Api/TeacherController.php</code>
12	S-03: attempt limits against OTP guessing (10 per minute per IP, 10 per hour per email) on <code>verify-otp</code> , <code>reset-password</code> and <code>login-otp/verify</code>	<code>AppServiceProvider.php</code> , <code>routes/api.php</code>
13	S-03 (web): the password-reset code is invalidated after 5 wrong attempts, safe comparison with <code>hash_equals</code> , and a limit on sending/verifying the code	<code>Web/UnifiedAuthController.php</code> , <code>routes/web.php</code>
14	S-03: replaced <code>rand()</code> with <code>random_int()</code> for OTP generation in 7 web controllers	<code>Web/*Controller.php</code>
15	S-04: <code>request-device-reset</code> is now under <code>throttle:login</code> ; limits added on <code>register</code> , <code>resend-otp</code> and <code>forgot-password</code>	<code>routes/api.php</code>
16	S-07: verify <code>X-Telegram-Bot-API-Secret-Token</code> on the webhook (<code>TELEGRAM_WEBHOOK_SECRET</code>); the request is rejected in production if not configured	<code>TelegramWebhookController.php</code> , <code>config/services.php</code>
17	S-06: a file-type allow-list (<code>mimes</code> :) on chat attachment upload (9 places), with the extension taken from the file content, not its name	<code>ChatController.php</code> and others

Actual test: 12 OTP guesses were sent; 10 were accepted for processing (wrong code, `400`) and the rest were rejected with `429`.

Verification: all migrations (about 120) were run on a new empty database and succeeded in full. The duplicate files are guarded by `hasColumn/hasTable` or add different columns, so they do not conflict. No migration was deleted on purpose.

Third batch (full remediation plan: [11-remediation-plan.md](#))

Method: every fix has an automated test written first. For most of them I proved that the test **fails on the old code and passes on the new**. Result: **104+ automated tests pass** (there were none).

#	Fix	Files
18	N-10 (critical): the head of department (web) can no longer delete or edit (email/password) any user; limited to teachers, students and parents of their department , never touching Admin/Affairs/other heads	<code>HODWebController</code> , <code>Support/Access</code>
19	N-11: deleting schedules/exams/reports, course weighting and leave requests limited to the head's department, with a leave state machine (no skipping the parent's approval)	<code>HODWebController</code>
20	N-08/N-09: the head's API (leave, reports, student services, course grades, meetings and parent summons) scoped to department and correct stage. The dead <code>updateLeaveStatus</code> that set <code>approved</code> directly was removed	<code>DepartmentHeadController</code> , <code>HODController</code> , <code>ParentMeetingController</code>
21	N-02/N-04/N-03/N-05/N-12: a teacher cannot answer other students' excuses, mark attendance for a lesson/student outside the course, read/submit others' reports, or summon parents of non-students; same in the web version	<code>TeacherController</code> , <code>TeacherWebController</code>
22	N-06/N-07/N-13: a student cannot read others' leave, submit assignments of non-enrolled courses, or be auto-enrolled in any course by opening its link (the materials page enrolled them instantly)	<code>StudentController</code> , <code>StudentWebController</code>

#	Fix	Files
2 3	N-15: role hierarchy: Affairs cannot edit/delete/disable Admin or other Affairs accounts (API + web)	AffairsController , AffairsWebController
2 4	Chat: the "who can message whom" matrix now applies to group creation (it was a bypass)	ChatController
2 5	S-08/S-09/D3/D6/S-16/S-17/D7: 30-day token expiry (app returns to login on 401), restricted CORS, fixed OTP removed (explicit dev-only option), 8-character passwords when set (server and app), CSRF exception removed, sessions encrypted by default	config/* , AuthController , bootstrap/app.php , Flutter
2 6	S-10/S-12: attendance no longer accepts faces implicitly (the fake 90/96 scores removed): first_time / suspicious with a teacher alert, future scan times rejected, ATTENDANCE_REQUIRE_FACE option	StudentController , config/attendance.php
2 7	S-13: face images in private storage with a permission-checked viewer, and a faces:secure command to move old ones	FaceImageStore , FaceImageController , SecureFaceImages
2 8	S-14: the Telegram scanner uses a signed, expiring link + an encrypted token that identifies the student (no client chat_id), and honest face statuses	TelegramWebhookController , TelegramBotHandler
2 9	S-19: server_url in the AI assistant restricted (same host / local network / allow-list)	AiAssistantController
3 0	B-01: student lookup by university ID for Admin/Head pages through a dedicated role-scoped web route (it always failed)	StudentLookupController
3 1	B-08/B-07 partial: ending a session no longer turns "late" into absent and honors approved leave; excused absences are excluded from warnings	TeacherController , AbsenceWarningService
3 2	B-11/F4: FCM access-token cache, and removal of the hard-coded assumption that user 1 is the admin	FcmService , Support/Access
3 3	Removed 6 dead controllers with no authorization (N-14, N-16)	Api/* , WebHead/*

Latent functional bugs found by the tests (each crashed whole features with a 500):

#	Defect	Impact	Fix
L-01	Code uses the non-existent column students.department_id	Teacher parent-summons, educator-students list and the head's summons list all returned 500	Join through programs.department_id
L-02	parent_summons.status is an enum that rejects pending_hod / pending_affairs used by the code	The whole summons flow was broken	New migration 2026_10_06_100000 widens the values
L-03	Code reads parent_summons.summon_id ; the real key is id	Forwarding/issuing a summon failed	Replaced with id
L-04	departments.name_ar and students.academic_year do not exist	Lists failed	departments.name and users.academic_year
L-05	TelegramWebhookController uses DB without importing it	Every Telegram-scanner attendance returned 500	Import added
L-06	Route /teacher/parent-summons/request pointed to a missing method	The teacher's parent-summon screen did not work	Bound to sendParentSummon
L-07	The end-session response said "assignment created"	Wrong message	Correct text

Important for deployment: run `php artisan migrate` to apply the summons migration (L-02) on your live database, and add the new variables from `.env.example`.

Withdrawn or intentionally not fixed:

Item	Decision
N-01 (grading a submission outside the teacher's course)	Withdrawn: my false alarm (the check exists past the first 17 lines). Its test is kept as a regression guard
S-11 (client-sent face embedding, no liveness)	Documented accepted risk: no complete fix without an architectural change (on-device attestation/Play Integrity). Mitigated by device binding, location, reject logging and teacher alerts
S-15 (QR shared with an absent student)	Accepted risk: mitigated by face, device and location checks
B-05 (two rows for the same lesson on two days)	Not changed: needs an academic decision
B-06 (<code>created_at</code> = client time)	Accepted: the scan time is now bounded by the session window (and never in the future)
B-07 (per-semester absence count)	Deferred: an academic decision for the institute
B-10 (synchronous FCM)	Partial: cost reduced by caching; queue sending requires <code>queue:work</code> , so it is left for the next phase
8-character password	Applied to self-service changes only; staff-created accounts remain at 6 (needs review of the creation screens)

3. Open security issues

🔴 Critical / high (all fixed)

S-01 ✅ A parent controls any student's permission requests (IDOR + broken authorization)

- **Status:** ✅ **Fixed**, see second batch (item 10).
- **Location:** `StudentParentController::respondPermission -> POST /api/parent/permissions/{requestId}/respond`
- **Problem:** it fetched the request by id only and did not check that the owner is a child of the logged-in parent. Any parent could accept or reject any student's permission by guessing `request_id` (sequential numbers).
- **Impact:** tampering with absence and permission records, and false notifications reaching students.
- **Fix:** verify the link through `parent_students` before any change.

S-02 ✅ Reading any student's data from a parent account

- **Status:** ✅ **Fixed**, see second batch (item 10).
- **Locations (all under `role:parent`):** `getFullPerformance`, `getAssignments`, `getPermissions` (`/api/parent/performance/{studentId}`, `/api/parent/student/{studentId}/assignments`, `/api/parent/student/{studentId}/permissions`)

- **Problem:** no check that `$studentId` is the parent's child. (By contrast `ParentController::getChildDetails` and its siblings check ownership correctly, so the right pattern already exists in the project.)
- **Impact:** leak of grades, assignments and absence permissions of any student.

S-03 Weak OTP verification (guessable)

- **Status:**  **Fixed:** limits, safe comparison and a secure generator (items 12 to 14). **Remaining warning:** the code fixes the OTP to `123456` when `APP_ENV=local` (`AuthController`, lines 591 and 1029). If a server is deployed with that setting, any account can be taken over. Production must use `APP_ENV=production`, and the exception should preferably be removed from the code.
- **Location:** `AuthController::verifyOtp` and `verifyLoginOtp`
- **Problem:** a 6-digit code (one million possibilities) with no limit on wrong attempts; `verifyLoginOtp` issues a full login token on success, so guessing meant account takeover (knowing the email was enough); generation used `rand()` in the web controllers.

S-04 `requestDeviceReset` bypassed account logout

- **Status:**  **Fixed** (item 15).
- **Problem:** it checked the password with `Hash::check` without `LoginThrottleGuard` or `throttle:login`, allowing password guessing without logout.

S-05 Any teacher controlled other teachers' attendance sessions and students' face data

- **Status:**  **Fixed** (item 11).
- **Problem:** `endSession`, `getSessionAttendance`, `refreshQrToken` and `resetStudentFace` fetched the record by id without checking it belongs to the logged-in teacher. A teacher could end another's session, which marks everyone who has not attended as absent and triggers warnings and parent summons through `AttendanceObserver`.

S-06 Chat uploads with a client-chosen extension

- **Status:**  **Fixed** (item 17). **Remaining on the server side:** block PHP execution inside `storage/` and `public/uploads/` (section 6).
- **Problem:** the extension came from `getClientOriginalExtension()` with no allow-list. `.php`, `.html` or `.svg` could be uploaded. If `storage` is served directly through a symlink on Apache/Nginx in production, remote code execution or stored XSS becomes possible.

S-07 Telegram webhook without source verification

- **Status:**  **Fixed** (item 16). **Required from you:** register the webhook with a `secret_token` and put the same value in `.env`.
- **Problem:** no check of `X-Telegram-Bot-API-Secret-Token`; anyone who knows the URL could send forged updates to the bot.

 **Medium (open)**

#	Item	Details
S-08 ✓ Fixed	Sanctum tokens never expire	<code>config/sanctum.php: 'expiration' => null</code> . A token is valid forever unless logged out. (<code>SingleSessionGuard</code> has logic that drops a 24-hour idle token, but only on a new login attempt)
S-09 ✓ Fixed	CORS fully open	<code>config/cors.php: allowed_origins => ['*']</code> with <code>allowed_methods => ['*']</code> . Must be restricted to the project domains in production
S-10 ✓ Fixed	Face check passes implicitly when there is no reference	<code>StudentController::scanAttendanceQr</code> : with no reference photo, <code>face_score = 96</code> and <code>verified</code> ; if vector extraction fails, <code>90</code> ; at first enrollment or after <code>requires_face_reset</code> , any face is accepted and stored as the reference (100%)
S-11 ✓	Face is compared on an embedding the client sends	The <code>face_embedding</code> is computed in the app and sent to the server. A modified app (or a direct HTTP request) can send the stored embedding itself. There is no server-side liveness check
S-12 ✓ Fixed	<code>scanned_at</code> , <code>device_id</code> and <code>latitude/longitude</code> come from the client	All are unsigned values the client sends. A student can forge location, device and time (within the session window). The offline sync policy <code>anytime</code> opens the door to late recording
S-13 ✓ Fixed	Face images in a public folder	Saved in <code>public/uploads/faces/</code> and reachable directly by URL. Sensitive biometric data. (The folder was added to <code>.gitignore</code> , but it is still public on the server)
S-14 ✓ Fixed	<code>POST /telegram/record-attendance</code> (web, public)	Identifies the student only by the <code>chat_id</code> sent in the request, with no login. Whoever knows a student's Telegram <code>chat_id</code> and holds a live QR token can record their attendance. The only protection is the optional face check (<code>face_image</code> and <code>face_embedding</code> are both nullable)
S-15 ✓	QR token	<code>Str::random(32)</code> (good). But it can be shared with an absent student within the session (mitigated by the face, device and location checks)
S-16 ✓ Fixed	CSRF protection partly disabled	<code>bootstrap/app.php: affairs/accounts</code> and <code>affairs/accounts/*</code> are excluded from CSRF. The reason must be reviewed, since these are account-modifying routes
S-17 ✓ Fixed	<code>/broadcasting/auth</code>	Defined twice (one public, one inside <code>affairs</code>). In <code>routes/channels.php</code> the <code>presence-online</code> channel uses <code>\$user->first_name</code> , which may be <code>null</code>
S-18 ⚠	<code>/web-notifications/*</code> (web)	Relies on <code>Auth::id()</code> only (good), but <code>delete</code> over <code>GET/POST</code> has no dedicated CSRF. Needs verification
S-19 ✓ Fixed	Link injection into AI assistant replies	<code>AiAssistantController::resolveServerBaseUrl</code> accepts <code>server_url</code> from the request body as is (any non-localhost domain) and places it in the login links inside the assistant replies. A logged-in user can make the assistant emit links pointing to a domain they choose (phishing). A domain allow-list is needed

🟡 Low / improvements

- ✓ **Self-registration:** `register` accepts `role` from `student|parent` only (good). Rate limits were added after this review (item 15).
- ✓ **Password:** minimum 6 characters, with no complexity rules.
- ✓ **SESSION_ENCRYPT=false** and **SESSION_DRIVER=file** (does not scale to several servers).
- ✓ **last_login** and **UserActivity:** record good events, but there is no alert when a password/email is changed.
- ✓ **Hard-coded** `sender_user_id = 1` in `AbsenceWarningService` (assumes user 1 is the admin).

4. Logic errors and potential defects

#	Item	Status
B-01	The admin <code>create_parent</code> and HOD <code>accounts</code> pages call <code>/api/student/info/{id}</code> with a web session, while the route requires a Sanctum token and a student/parent role, so the call most likely fails (401)	✓
B-02	Duplication and dead code: <code>Api\MessageController</code> is not attached to any route (chat works through <code>ChatController</code>); <code>/system/settings</code> and <code>/broadcasting/auth</code> were defined twice; Affairs routes sit inside the <code>auth:sanctum</code> group and repeat the same middleware	✓
B-03	The <code>users.role</code> (enum) column, <code>role_id</code> and <code>roles</code> coexist. Values may not match (the original <code>enum</code> does not include <code>affairs</code>)	✓
B-04	Link keys are inconsistent: <code>parent_students.parent_id</code> and <code>student_id</code> may point to <code>user_id</code> or to <code>parent_id/student_id</code> (some queries tolerate both with <code>whereIn</code> , others do not)	✓
B-05	<code>Attendance::updateOrCreate</code> is keyed by (student, lesson, date). Scanning the QR twice on two days for the same lesson creates two rows	⚠
B-06	<code>\$attendance->created_at = \$scannedAt</code> overwrites the creation time with a client value, which pollutes the audit trail	✓
B-07	<code>AbsenceWarningService</code> : counts "absence days" across all courses and semesters together, never resetting between semesters; each warning is guarded by <code>exists()</code> and never repeats for the same student for life	✓
B-08	<code>endSession</code> records absence even for someone with an approved excuse (<code>excuse_status</code>). Needs verification	⚠
B-09	Disappearing chat messages are deleted by a scheduled task (<code>everyFiveMinutes</code>), which requires <code>schedule:run</code> to actually run on the server (cron)	✓
B-10	<code>SendFcmNotificationJob</code> exists, but <code>FcmService::sendToUser</code> sends synchronously inside the request, so an FCM delay slows the API	✓
B-11	<code>FcmService::getAccessToken</code> builds a JWT and requests an access token on every send without caching	✓
B-12	<code>TeacherController.php</code> and some files have Arabic comments with corrupted encoding (mojibake) . The code works, but the comments are unreadable	✓
B-13	Raw queries (<code>DB::raw/whereRaw/selectRaw</code>) in 21 files (136 places). No confirmed injection was found in the sampled code, but every place that includes user input must be reviewed	⚠
B-14	<code>{!! !!}</code> in Blade: only 6 places, and the two examined pass through <code>e()</code> (safe). The remaining search hits are legitimate uses	✓
B-15	The absence of N+1 queries is unverified; the large controllers (<code>AffairsWebController</code> ~189KB) have loops and nested queries that may be problematic	⚠
B-16	<code>face_embedding</code> is stored as JSON with 192 numbers per student and updated with a moving average (0.85/0.15) on each success, so the reference may gradually drift	⚠
B-17	Flutter app: local server addresses are hard-coded (<code>127.0.0.1</code> and 3 LAN addresses) with handling for an old IP (<code>82.137.250.43</code>). There is no environment configuration (dev/staging/prod)	✓
B-18	Flutter app: 2-second chat polling (in parallel with Pusher) and 30-second notification polling consume battery, data and server capacity, and are why <code>throttle:api</code> is set at 240 requests/minute	✓

5. Code quality and maintenance (technical debt)

Item	Detail
Very large controllers	AffairsWebController 189KB, TeacherController 144KB, AffairsController 130KB, StudentController 126KB, TeacherWebController 104KB. Business logic, view logic and queries in a single file
Web/API duplication	The same function is written twice (e.g. <code>scanAttendanceQr</code> in API, Telegram and Web) and the face code (<code>extractImageVector</code>) is duplicated between <code>StudentController</code> and a trait
Tests	<code>tests/</code> contains only <code>ExampleTest</code> . Flutter has the default <code>widget_test.dart</code> . Effectively zero coverage
Code documentation	Many comments, some documenting important decisions (good), but some mixed with a development tone
Migrations	About 120, some duplicated or conflicting in name (<code>.php.php</code>). They run from scratch, but squashing later is advised
Misplaced files	<code>render_*.cjs</code> , <code>start_bot.*</code> and <code>run_migrations.bat</code> at the root; the <code>_deprecated/</code> and <code>سيرفر/</code> folders
Dependencies	<code>barryvdh/laravel-dompdf</code> and <code>carlos-meneses/laravel-mpdf</code> together (two PDF libraries) plus <code>maatwebsite/excel</code> . Check whether both are needed
Naming	<code>app/Http/Middleware/Edu_Pridge_flutter.code-workspace</code> : a VSCode workspace file inside the Middleware folder (wrong place)
README	Default in both projects

6. Production environment checklist

The current `.env` settings are for local development and must be changed:

- ☐ `APP_ENV=production` and `APP_DEBUG=false` (currently `local` and `true`, which exposes error details)
- ☐ `MAIL_MAILER` is currently `log`, so OTP emails are **not actually sent** (they are only written to the log). Configure real SMTP
- ☐ `SESSION_SECURE_COOKIE=true` and `SESSION_ENCRYPT=true`
- ☐ `SANCTUM_EXPIRATION` set, and `CORS` restricted to specific domains
- ☐ Run `php artisan schedule:run` through cron (for disappearing messages, the attendance summary and log cleanup)
- ☐ Run a queue worker `queue:work` (currently `QUEUE_CONNECTION=database`)
- ☐ Configure the Telegram webhook with a `secret_token`, or run `TelegramPoll` as a service
- ☐ File `storage/app/firebase-service-account.json` for FCM sending (not in the repository, which is correct)
- ☐ Keys: `GEMINI_API_KEY` (added to `.env.example`), `PUSHER_*` and `TELEGRAM_BOT_TOKEN`
- ☐ Block PHP execution inside `storage/` and `public/uploads/`, and move `uploads/faces` out of the public root
- ☐ Mandatory HTTPS (the code forces it via `URL::forceScheme` when `production`)
- ☐ Regular database backups (the `ExportDatabase` command exists)
- ☐ In the Flutter app: build flavors for the server address, and remove the hard-coded LAN addresses

7. What was not reviewed (limits of this study)

- Not every controller was read line by line. The following need an authorization review: `AdminController`, `AffairsController`, `DepartmentHeadController` (head scope: does a head see only their department?), `HODController`, `ParentMeetingController`, `Api\MessageController` and `NotificationController`.
 - The grade and weight calculation logic (`StudentAcademicService`) was not reviewed for the correctness of the academic calculations.
 - No performance test or dynamic penetration test was done.
 - The Flutter code was not reviewed in depth (~73K lines of screens); the focus was on services and structure.
 - The license of the `mobile_face_net.tflite` model must be confirmed (`assets/models/NOTICE.md`) before any commercial use.
-

8. Recommended fix order

1. ~~Week 1 (S-01 to S-07)~~ **Done**.
2. ~~Medium items (S-08 to S-19) and the extended authorization (N-xx)~~ **Done** (third batch).
3. **Next:** the section 6 items (production setup), institute decisions (B-05/B-07), liveness checking (S-11), and splitting the controllers.
4. **Weeks 3 to 4:** write Feature tests for the authorization routes (for each role: can it reach another's data?) so these errors do not return.
5. **Later:** split controllers into Services, unify web/API logic, and squash the migrations.

Remediation Plan

Status:  **Executed** on branch `fix/security-hardening-phase2` (see section 6) **Prepared:** 2026-10-06

Reference: the vulnerabilities and bugs in [08-project-status.md](#) + the extended inventory (section 1)

1) Extended inventory result

After phase one (S-01 to S-07) I inventoried every function that takes an `{id}` from the request:

Metric	Value
Functions taking an id (API + Web)	222
No ownership marker (automatic scan)	37
Only one marker	40
Flagged and reviewed by hand	the most dangerous, and most teacher/student/head functions

The automatic scan is a **detector, not a proof**: everything flagged is reviewed by hand, and what was not flagged may still hide a defect. That is why the plan includes **automated authorization tests** that lock the result in.

Defect patterns found (new):

Code	Defect	Impact
~~N-01~~	~~ <code>gradeAssignment</code> grades any submission~~ Withdrawn: my false alarm (the check exists); its test is kept as a regression guard	-
N-02	<code>respondAbsenceRequest</code> answers any absence request (reading was limited, answering was not)	Tampering with excuses
N-03	<code>getStudentAcademicStats</code> and <code>submitEvaluation</code> do not check the report request belongs to this teacher	Reading academic data and writing someone else's evaluation
N-04	<code>markAttendance</code> checks the course but not that <code>lesson_id</code> and <code>student_id</code> belong to it	Attendance for a lesson or student outside the course
N-05	<code>sendParentSummon</code> lets any teacher summon any student's parent	False summons
N-06	<code>StudentController::getLeaveDetails</code> does not check the request is the student's own	Reading others' leave reasons
N-07	<code>submitAssignment</code> does not check enrollment in the assignment's course	Submitting to assignments outside the student's courses
N-08	Head of department (API): leave, notes, report deletion/sending, student services and course grades not limited to the department	A head controls another department's requests
N-09	<code>HODController::updateLeaveStatus</code> set <code>approved</code> directly, skipping the parent's approval and the Affairs sequence	Leave without parental approval
N-10	<code>HODWebController::deleteAccount</code> deletes any user (even Admin) and <code>updateAccount</code> edits any account	Critical: a head deletes the Admin or sets their password

Code	Defect	Impact
N-11	<code>HODWebController: deleteSchedule, deleteExam, deleteReport, updateHodNotes, updateCourseWeight, updateLeaveStatus</code> unscoped	Cross-department tampering
N-12	<code>TeacherWebController: endSession, refreshSessionQr, getAbsentees, exportAttendance</code> without ownership (the web twin of S-05)	Same impact as S-05
N-13	<code>StudentWebController::downloadLesson</code> and <code>courseMaterials</code> with no enrollment check (and the latter auto-enrolls)	Downloading and self-enrolling in any course
N-14	<code>ScheduleController</code> and <code>ExamScheduleController</code> (<code>update / destroy</code>) unscoped	Editing any department's timetable
N-15	No role hierarchy: Affairs/Head can edit accounts of higher roles	Privilege escalation
N-16	<code>ParentProfileController</code> not attached to any route (dead code)	Removed

2) Principles

1. **Test before fix:** every defect gets a Feature test that fails before the fix and passes after.
2. **One place for authorization:** a central `App\Support\Access` helper instead of repeated checks.
3. **Deny by default:** nobody reaches a resource without proving ownership or role.
4. **One commit per group:** clear message, tests run before each commit.
5. **Do not break the app:** any change affecting Flutter behavior is made in the app in the same batch.
6. **Honest documentation:** `08-project-status.md` is updated with what was actually fixed and what remains.

3) Workstreams

WS-A: Foundation

#	Task	Acceptance
A 1	Test infrastructure: a dedicated MySQL test database, a guard that refuses to run on a database not ending in <code>_test</code> , data helpers	<code>php artisan test</code> runs and never touches the real database
A 2	<code>App\Support\Access</code> : parent/teacher/head ownership, course enrollment, role hierarchy	Used by all following fixes
A 3	CI on GitHub Actions (MySQL + <code>php artisan test</code>)	Passes on every push

WS-B: API authorization (N-01 to N-09, N-14 to N-16)

Limit teachers to their courses and students, students to themselves and their courses, heads to their department, and enforce a **state machine** for leave requests (no jumping stages).

WS-C: Web authorization (N-10 to N-13)

The same rules through the same central helper. **N-10** (account delete/edit) first.

WS-D: Authentication, sessions and settings

#	Task
D1	Sanctum token expiry (default 30 days, configurable) + automatic logout in Flutter on 401
D2	Restrict CORS to domains configured in .env
D3	Remove the fixed OTP 123456 (now an explicit dev-only OTP_FIXED_CODE option)
D4	Review and remove the CSRF exception for affairs/accounts*
D5	presence-online payload
D6	Password policy (8 characters minimum when set)
D7	SESSION_ENCRYPT and safe defaults

WS-E: Attendance and biometric integrity (S-10 to S-14, B-05, B-06, B-08)

#	Task
E 1	No implicit face acceptance: a missing reference/embedding is recorded suspicious and flagged to the teacher (instead of verified with 90/96)
E 2	Bound scanned_at : never in the future, within the session window
E 3	Move face images to private storage, served through a permission-checked route
E 4	Telegram: attendance from the scanner requires a signed, expiring link issued by the bot, instead of trusting chat_id alone
E 5	endSession respects approved excuses
E 6	S-11: no complete server-side fix (an architectural decision). Documented as an accepted risk with mitigations: rate limiting, device binding, reject log. A liveness check stays on the roadmap

WS-F: Remaining items

#	Task
F1	S-19: allow-list for the server_url domain in the AI assistant
F2	B-01: fix the /api/student/info call from web pages (a proper web route with session authorization)
F3	B-10/B-11: send FCM through the queue + cache the access token
F4	The hard-coded sender_user_id = 1 in warnings becomes the first real admin
F5	B-13: review raw queries that mix user input, and convert them to bindings

WS-G: Verification and handover

Run all tests, regenerate the documentation (docs/tools), update 08-project-status.md , the PDF and the site, review the final diff, and prepare a change summary.

4) Decisions I made (adjustable assumptions)

Decision	Choice	Reason
Token lifetime	30 days	Balance between security and not bothering students
Face when no reference exists	Accept with a suspicious flag (not reject)	We do not break new students before their first enrollment

Decision	Choice	Reason
Minimum password	8 characters	Standard
Absence counting per semester (B-07)	Deferred	An academic decision (do warnings reset each semester?) for the institute
S-15 (QR sharing) and S-11	Documented accepted risk	No complete fix without an architectural change

5) Risks and rollback

- Every group is in its own commit, so rollback is `git revert <commit>`.
- Changes are tried on a separate test database, and no destructive migration is run.
- Fixes that may change user behavior (token expiry, the signed Telegram link) are flagged in the status file and should be tried on a device before deployment.

6) Execution log

Workstream	Status	Notes
WS-A Foundation	✓	Separate test database (<code>edu_bridge_test</code>) with a guard against running on a database not ending in <code>_test</code> ; <code>App\Support\Access</code> ; test data helper. (Not done yet: CI on GitHub Actions)
WS-B API authorization	✓	N-02 to N-09, N-14 to N-16. Head/teacher/Affairs tests were proven to fail on the old code
WS-C Web authorization	✓	N-10 to N-13 + role hierarchy + chat groups
WS-D Authentication and settings	✓	D1 (30-day token + Flutter), D2 (CORS), D3 (fixed OTP removed), D4 (CSRF), D5, D6 (8 characters on self-service), D7
WS-E Attendance	✓	E1, E2, E3, E4, E5; E6 is a documented accepted risk
WS-F Remaining items	✓	F1 to F5 (F3: cache only; queue sending deferred)
WS-G Verification and handover	✓	105+ tests pass ; documentation regenerated; status files, PDF and site updated

Not done, and why

- **CI (A3)**: needs GitHub repository settings, which belong to you or the repository owner.
- **B-07** (per-semester absence counting): an academic decision.
- **S-11** (liveness): an architectural decision, outside what a server fix can do.
- **8-character limit for staff-created accounts**: needs a review of the creation screens in the app.
- **Splitting the controllers**: a separate project (phase 3 of the roadmap).

Latent functional bugs found by the tests

See the L-01 to L-07 table in [08-project-status.md](#): whole features were crashing with a 500 (teacher parent-summons, the Telegram scanner, the head's lists...).

Proposed Roadmap

In priority order. The numbers (S-xx / B-xx) refer to items in [08-project-status.md](#).

Phase 1: Stabilize before any launch (1 to 2 weeks)

Item	Description	Effort
Production setup	<code>APP_ENV=production</code> , real SMTP, cron, Supervisor for the queue, HTTPS, block PHP execution in storage	1 day
S-08	Expire Sanctum tokens (<code>SANCTUM_EXPIRATION</code>)	hours
S-09	Restrict CORS to the project domains	hours
S-13	Move face images out of the public root and serve them through a protected route	1 day
S-14	Authenticate <code>/telegram/record-attendance</code> and bind <code>chat_id</code> to a verified session	1 day
S-19	Domain allow-list for <code>server_url</code> in the AI assistant	hours
Remove the <code>123456</code> OTP exception from the code	Remove the <code>environment('local')</code> condition	1 hour
Review the remaining controllers	Admin, Affairs and head of department (see section 7 of project status)	2 to 3 days

Phase 2: Tests and quality (2 to 4 weeks)

- Authorization Feature tests:** for each role, make sure it cannot reach another's data (prevents S-01/S-02/S-05 from returning). This has the highest return.
- Tests for the attendance logic (`scanAttendanceQr`), warnings (`AbsenceWarningService`) and grade calculation (`StudentAcademicService`).
- Simple CI (GitHub Actions): `composer install`, `php artisan test`, `pint`, and `flutter analyze`.
- Set up Flutter environments (`--dart-define`) instead of hard-coded server addresses (B-17).

Phase 3: Reduce technical debt (1 to 3 months)

Item	Description
Split controllers	Move logic into Services/Actions, starting with the largest: <code>AffairsWebController</code> , <code>TeacherController</code> , <code>AffairsController</code> , <code>StudentController</code>
Unify duplicated logic	Attendance recording (API/Web/Telegram), face vector extraction, absence-day counting
Unify link identifiers	Settle <code>parent_students</code> and <code>leave_requests.student_id</code> on <code>user_id</code> or the internal id (B-04)
Unify <code>role</code> and <code>role_id</code>	Rely on <code>role_id</code> only (B-03)
Squash migrations	Squash into one baseline schema (it already runs from scratch)
FormRequests	Move validation out of controllers (some exist in <code>Requests/Admin</code>)

Item	Description
Remove dead code	Api\MessageController, the chats, user_activity, session and otps tables, and the _deprecated folder
FCM	Send through the queue (SendFcmNotificationJob) and cache the access token (B-10/B-11)
Warnings	Count absences per semester (B-07)

Phase 4: Features and improvements (open-ended)

- **Stronger attendance:** on-device liveness detection with request signing, or detection of tampered devices (Play Integrity).
- **Replace polling with WebSocket only** after Pusher is stable (saves battery and server load).
- **Monitoring:** Sentry/central logging, and alerts for queue failures.
- **Interactive API documentation:** OpenAPI/Swagger (currently a table generated from the routes).
- **Multi-institution:** if the goal is to sell the product to more than one institute, data separation (multi-tenancy) and per-institution settings are required.
- **Performance:** review N+1 queries in the large controllers (B-15), and add indexes as measured.
- **Privacy:** a retention policy for biometric data (faces), explicit consent, and deletion at graduation.

Readiness criteria for sale/handover

- [] Phase 1 complete
- [] Authorization tests (phase 2.1) pass
- [] Running 06-setup-and-deploy.md on a clean machine ends with a working system
- [] The face-model license is settled
- [] The handover checklist 10-handover-checklist.md is satisfied

Handover Checklist

For use when transferring the project to a buyer or a new developer. Tick each item when done.

1) Code and repository

- ☐ Both repositories (Laravel and Flutter) in their final version, with the approved branches explained
- ☐ Merge the security-fix branch (`chore/security-cleanup-and-docs`) or document why it was not merged
- ☐ The repository is free of large and sensitive files (`ngrok.exe` , `.sql` files, `accounts.xlsx` , `*creds*` files)
- ☐ Review `.gitignore` in both projects
- ☐ Review git history: did any secrets enter it? (the data is said to be sample data, but check keys)
- ☐ Decide the new repository owner, and transfer ownership or hand over a copy (fork/export)

2) Accounts and keys (transferred, or created by the buyer)

Service	Use	Transfer?	Needed
Firebase (project <code>edu-bridge-246fd</code>)	FCM push and Firebase Web	☐ transfer ownership / ☐ new project	<code>google-services.json</code> , the service account, the web keys in <code>main.dart</code>
Pusher	Real-time chat	☐ / ☐	App ID/Key/Secret/Cluster (the public key and <code>cluster</code> are written in <code>chat_service.dart</code>)
Telegram Bot	The bot and OTP	☐ / ☐	A new token from BotFather, and <code>TELEGRAM_WEBHOOK_SECRET</code>
Google Gemini	AI assistant	☐ / ☐	<code>GEMINI_API_KEY</code>
SMTP	OTP emails	☐ / ☐	Mail credentials
Hosting / domain	Production	☐	Not currently in the repository
Google Play / App Store	Publishing the app	☐	Publisher accounts and the signing key

Important: every current key must be **rotated or replaced** at handover, so the previous owner keeps no access to the new system.

3) Data

- ☐ No real data in any copy that is handed over (all data is said to be sample data: check `*.sql` files and local backups)
- ☐ Seeders produce a complete demo environment (roles, students, courses, attendance, grades)
- ☐ Demo accounts are documented with passwords that are changed at handover
- ☐ Face embeddings and photos: wipe them from any sample copy

4) Licenses and intellectual property

- [] `mobile_face_net.tflite` **model**: license not explicitly documented (`assets/models/NOTICE.md`). Re-derive it from `sirius-ai/MobileFaceNet_TF` (Apache-2.0) or obtain written confirmation
- [] Review the licenses of Composer, pub and npm packages (`composer licenses` , `flutter pub deps`)
- [] Fonts, images and logos (`public/images/logos` : `dtc` , `unrwa` , others): do you have the right to hand them over? (they may belong to third parties)
- [] **Project rights**: if this is a university graduation project or was built for an organization, make sure the university/supervisor/organization agrees to the transfer of ownership. (An ownership-transfer contract, an NDA and a legal review are recommended)
- [] Credit the contributors (the development team)

5) Documentation and knowledge

- [] `docs/` is complete in Arabic and English
- [] Try `06-setup-and-deploy.md` literally on a clean machine
- [] A recorded or written walkthrough of the most important flows
- [] The known-issues list (`08-project-status.md`) is updated on handover day
- [] A named contact for post-handover questions, and a support period if any

6) Production (if a system is running)

- [] A full backup of the database and files
- [] Server configuration (Nginx/cron/Supervisor) documented
- [] Transfer the domain and HTTPS certificates
- [] Stop or transfer any tunnels (ngrok) and auto-start services on the developer's machine (`سيرفر / .bat`)

Appendices (auto-generated)

Database Dictionary

Auto-generated from the real migrations (run `php artisan migrate` on an empty database, then read `information_schema`). **62 tables and 3 views, 579 columns, 72 foreign keys.**

Read this first

- **Primary keys are custom, not `id`**, in most tables: `user_id`, `student_id`, `teacher_id`, `course_id`, `lesson_id` ... (exceptions: `programs.id`, `grade_events.id`, `attendance_sessions.id`).
- **There are more logical relations than declared foreign keys** (only 72 FKs). Much of the linking is done in code with no database constraint, e.g. `parent_students` and `leave_requests.student_id`.
- **Consistency warning:** `parent_students.parent_id/student_id` and `leave_requests.student_id` may hold `users.user_id` or the internal id (`parents.parent_id` / `students.student_id`). The code handles both (see `08-project-status.md`, item B-04).
- Legacy tables still present: `chats`, `user_activity`, `session`, `otps`, `subjects`. They appear unused (verify before dropping).

Relationship diagram (core tables)

Shows the most important tables and relations (declared foreign keys plus logical relations documented from the models). Mermaid format, rendered automatically on GitHub and the documentation site.

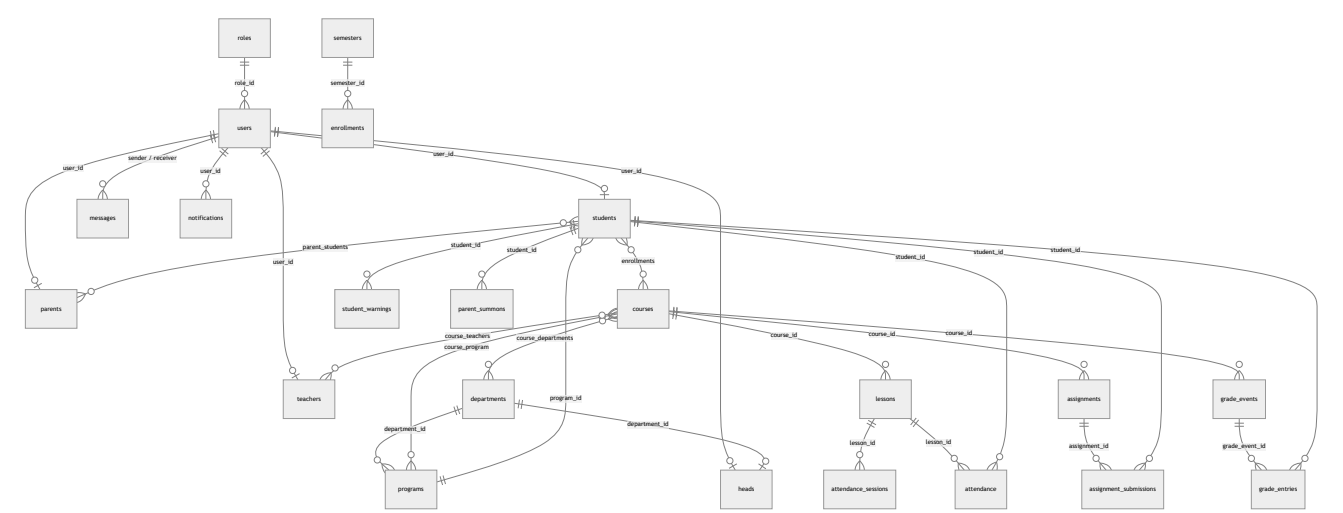


Table index

Table	Type	Description
<code>admin_generated_reports</code>	Table	Log of reports generated by the administration
<code>admin_profile_stats_view</code>	View	View: admin profile statistics

Table	Type	Description
<code>admins</code>	Table	Administrator profile
<code>affairs_dashboard_stats_view</code>	View	View: Affairs dashboard statistics
<code>announcements</code>	Table	Announcements and activities (target audience, images, link, event details)
<code>assignment_submissions</code>	Table	Student submissions and grading
<code>assignments</code>	Table	Assignments
<code>attendance</code>	Table	Attendance log: status, device, location, face match score, reject reason, excuse
<code>attendance_sessions</code>	Table	Attendance session: rotating QR token, location and radius, validity
<code>cache</code>	Table	Laravel cache
<code>cache_locks</code>	Table	Cache locks
<code>calendar_events</code>	Table	Academic calendar events
<code>chats</code>	Table	Legacy chat table
<code>course_program</code>	Table	Course-to-program links
<code>course_teachers</code>	Table	Teachers assigned to courses, with a role (e.g. advisor)
<code>courses</code>	Table	Courses (title, code, hours, weight, year, semester)
<code>departments</code>	Table	Academic departments, plus the offline attendance sync policy (<code>offline_sync_policy</code>)
<code>enrollments</code>	Table	Student enrollment in courses per semester
<code>exams</code>	Table	Exam timetable
<code>failed_jobs</code>	Table	Failed jobs
<code>grade_entries</code>	Table	Per-student scores for each assessment event
<code>grade_events</code>	Table	Assessment events (exam / quiz / oral) with date, time and maximum score
<code>grade_report_requests</code>	Table	Head-of-department requests for grade reports from teachers
<code>grades</code>	Table	Exam grades (legacy system)

Table	Type	Description
group_user	Table	Group members
groups	Table	Chat groups
head_schedule_entries	Table	Timetable entries created by the head of department
heads	Table	Heads of department and their department link
job_batches	Table	Job batches
jobs	Table	Queue jobs
leave_requests	Table	Leave requests (student -> parent -> head -> affairs)
lessons	Table	Lectures / sessions (file, video, teacher, course); one attendance session is created per lesson
messages	Table	Chat messages (attachments, reply, forward, delete, disappearing, delivery/read)
migrations	Table	-
notifications	Table	In-system notifications (type, category, related id)
otp_codes	Table	Email verification codes (current)
otps	Table	Verification codes (legacy)
parent_meeting_requests	Table	Meeting requests from parents
parent_students	Table	Parent-to-student links (many-to-many)
parent_summons	Table	Parent summons (manual, or automatic after 10 absence days)
parents	Table	Parent profile
performance_reports	Table	Performance reports
personal_access_tokens	Table	Sanctum tokens
photo_change_requests	Table	Requests to change the student reference photo
programs	Table	Programs / specializations belonging to departments
quiz_options	Table	Quiz question options

Table	Type	Description
quiz_questions	Table	Quiz questions
quizzes	Table	Online quizzes
report_requests	Table	Behavioral report requests (from head or parent) and head notes
resources	Table	Course files and references
roles	Table	The six role definitions
schedules	Table	Weekly class timetable (day, time, room, class group)
semesters	Table	Academic semesters and the active-semester flag
session	Table	Legacy sessions table
student_requests	Table	Student service requests (appeal, documents, make-up exam, device unlock)
student_warnings	Table	Automatic absence warnings (first / second / final)
students	Table	Student profile: code, level, program, device binding, reference face embedding
system_settings	Table	System settings (theme)
teacher_dashboard_stats_view	View	View: teacher dashboard statistics
teachers	Table	Teacher profile, plus advisor fields (advisor_branch / advisor_year)
university_ids	Table	Pre-registered university IDs created by Affairs before a student registers
user_activities	Table	User activity log (audit)
user_activity	Table	Legacy activity log
users	Table	Core accounts for all roles (login, role, status, session, account lockout, FCM token, Telegram id)

admin_generated_reports

Log of reports generated by the administration

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
title	varchar(255)	No			

Column	Type	Null	Default	Key	Notes
report_type	varchar(255)	No			
department_id	bigint(20) unsigned	Yes	NULL		
department_name	varchar(255)	Yes	NULL		
program_id	bigint(20) unsigned	Yes	NULL		
program_name	varchar(255)	Yes	NULL		
semester_id	bigint(20) unsigned	Yes	NULL		
semester_name	varchar(255)	Yes	NULL		
from_date	date	Yes	NULL		
to_date	date	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

admin_profile_stats_view

View: admin profile statistics

Column	Type	Null	Default	Key	Notes
total_users	bigint(21)	Yes	NULL		
total_courses	bigint(21)	Yes	NULL		

admins

Administrator profile

Column	Type	Null	Default	Key	Notes
admin_id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: user_id -> users.user_id

affairs_dashboard_stats_view

View: Affairs dashboard statistics

Column	Type	Null	Default	Key	Notes
total_students	bigint(21)	Yes	NULL		
total_teachers	bigint(21)	Yes	NULL		
total_staff	bigint(21)	Yes	NULL		
pending_leaves	bigint(21)	Yes	NULL		
total_users	bigint(21)	Yes	NULL		

announcements

Announcements and activities (target audience, images, link, event details)

Column	Type	Null	Default	Key	Notes
announcement_id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
title	varchar(255)	No			
content	text	No			
image_path	varchar(255)	Yes	NULL		
category	varchar(255)	Yes	'general'		
image	varchar(255)	Yes	NULL		
images	longtext	Yes	NULL		
link_url	varchar(255)	Yes	NULL		
type	enum('general', 'course_specific')	No	'general'		
target_audience	varchar(255)	No	'all'	INDEX	
target_role	varchar(255)	Yes	NULL		
department_id	bigint(20) unsigned	Yes	NULL	INDEX	
academic_year	varchar(255)	Yes	NULL		
course_id	bigint(20) unsigned	Yes	NULL	INDEX	
created_at	timestamp	Yes	NULL	INDEX	
updated_at	timestamp	Yes	NULL		
event_date	date	Yes	NULL		
event_time	time	Yes	NULL		
location	varchar(255)	Yes	NULL		

Foreign keys: user_id -> users.user_id, course_id -> courses.course_id

assignment_submissions

Student submissions and grading

Column	Type	Null	Default	Key	Notes
submission_id	bigint(20) unsigned	No		PK	auto-increment
assignment_id	bigint(20) unsigned	No		INDEX	
student_id	bigint(20) unsigned	No		INDEX	
file_path	varchar(255)	Yes	NULL		
solution_text	text	Yes	NULL		
student_notes	text	Yes	NULL		
grade	decimal(5,2)	Yes	NULL		
feedback	text	Yes	NULL		

Column	Type	Null	Default	Key	Notes
submitted_at	datetime	No			
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id, assignment_id -> assignments.assignment_id

assignments

Assignments

Column	Type	Null	Default	Key	Notes
assignment_id	bigint(20) unsigned	No		PK	auto-increment
course_id	bigint(20) unsigned	No		INDEX	
teacher_id	bigint(20) unsigned	Yes	NULL	INDEX	
title	varchar(255)	No			
description	text	Yes	NULL		
notes	text	Yes	NULL		
file_path	varchar(255)	Yes	NULL		
file_name	varchar(255)	Yes	NULL		
file_type	varchar(255)	Yes	NULL		
due_date	datetime	No			
max_points	int(11)	No	100		
attachment_path	varchar(255)	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: teacher_id -> teachers.teacher_id, course_id -> courses.course_id

attendance

Attendance log: status, device, location, face match score, reject reason, excuse

Column	Type	Null	Default	Key	Notes
attendance_id	bigint(20) unsigned	No		PK	auto-increment
student_id	bigint(20) unsigned	No		INDEX	
lesson_id	bigint(20) unsigned	No		INDEX	
status	enum('present', 'absent', 'late')	No			
device_id	varchar(255)	Yes	NULL		معرف الجهاز الذي سجل الحضور

Column	Type	Null	Default	Key	Notes
latitude	decimal(10,7)	Yes	NULL		خط عرض موقع الطالب لحظة المسح
longitude	decimal(10,7)	Yes	NULL		خط طول موقع الطالب لحظة المسح
reject_reason	enum('expired_qr', 'device_mismatch', 'location_too_far', 'already_marked', 'session_closed', 'face_mismatch')	Yes	NULL		
face_image	mediumtext	Yes	NULL		
face_score	double	Yes	NULL		
face_status	enum('first_time', 'verified', 'suspicious', 'rejected')	Yes	NULL		
attendance_date	date	No			
excuse_text	text	Yes	NULL		
excuse_attachment	varchar(255)	Yes	NULL		
excuse_status	enum('none', 'pending', 'approved', 'rejected')	No	'none'		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id, lesson_id -> lessons.lesson_id

attendance_sessions

Attendance session: rotating QR token, location and radius, validity

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
lesson_id	bigint(20) unsigned	No		INDEX	
qr_token	varchar(255)	No		UNIQUE	
expires_at	timestamp	No	current_timestamp()		
session_expires_at	timestamp	Yes	NULL		
is_active	tinyint(1)	No	1		

Column	Type	Null	Default	Key	Notes
closed_at	timestamp	Yes	NULL		وقت إغلاق الجلسة من المعلم
latitude	decimal(10,7)	Yes	NULL		خط عرض موقع المعلم عند فتح الجلسة
longitude	decimal(10,7)	Yes	NULL		خط طول موقع المعلم عند فتح الجلسة
radius_meters	smallint(5) unsigned	No	50		الحد الأقصى للمسافة المسموح بها بالمترا (افتراضي 50م)
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: lesson_id -> lessons.lesson_id

cache

Laravel cache

Column	Type	Null	Default	Key	Notes
key	varchar(255)	No		PK	
value	mediumtext	No			
expiration	int(11)	No		INDEX	

cache_locks

Cache locks

Column	Type	Null	Default	Key	Notes
key	varchar(255)	No		PK	
owner	varchar(255)	No			
expiration	int(11)	No		INDEX	

calendar_events

Academic calendar events

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
title	varchar(255)	No			
event_date	date	No			
event_time	time	Yes	NULL		
location	varchar(255)	Yes	NULL		
department_id	bigint(20) unsigned	Yes	NULL	INDEX	
created_at	timestamp	Yes	NULL		

Column	Type	Null	Default	Key	Notes
updated_at	timestamp	Yes	NULL		

Foreign keys: user_id -> users.user_id, department_id -> departments.department_id

chats

Legacy chat table

Column	Type	Null	Default	Key	Notes
chat_id	bigint(20) unsigned	No		PK	auto-increment
sender_id	bigint(20) unsigned	No		INDEX	
receiver_id	bigint(20) unsigned	No		INDEX	
content	text	No			
sent_at	timestamp	No	current_timestamp()		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: sender_id -> users.user_id, receiver_id -> users.user_id

course_program

Course-to-program links

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
course_id	bigint(20) unsigned	No		INDEX	
program_id	bigint(20) unsigned	No		INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: program_id -> programs.id, course_id -> courses.course_id

course_teachers

Teachers assigned to courses, with a role (e.g. advisor)

Column	Type	Null	Default	Key	Notes
course_teacher_id	bigint(20) unsigned	No		PK	auto-increment
course_id	bigint(20) unsigned	No		INDEX	
teacher_id	bigint(20) unsigned	No		INDEX	
role	varchar(255)	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: `course_id` -> `courses.course_id`, `teacher_id` -> `teachers.teacher_id`

courses

Courses (title, code, hours, weight, year, semester)

Column	Type	Null	Default	Key	Notes
<code>course_id</code>	<code>bigint(20) unsigned</code>	No		PK	auto-increment
<code>code</code>	<code>varchar(50)</code>	Yes	NULL		رمز المادة الدراسية
<code>title</code>	<code>varchar(255)</code>	No			
<code>weight</code>	<code>int(11)</code>	No	1		تثقيل المادة (عدد الساعات)
<code>description</code>	<code>text</code>	Yes	NULL		
<code>level</code>	<code>varchar(255)</code>	No			
<code>hours</code>	<code>int(11)</code>	No	0		
<code>year</code>	<code>tinyint(4)</code>	Yes	1		
<code>semester_id</code>	<code>bigint(20) unsigned</code>	Yes	NULL	INDEX	
<code>created_at</code>	<code>timestamp</code>	Yes	NULL		
<code>updated_at</code>	<code>timestamp</code>	Yes	NULL		

Foreign keys: `semester_id` -> `semesters.semester_id`

departments

Academic departments, plus the offline attendance sync policy (`offline_sync_policy`)

Column	Type	Null	Default	Key	Notes
<code>department_id</code>	<code>bigint(20) unsigned</code>	No		PK	auto-increment
<code>name</code>	<code>varchar(255)</code>	No			
<code>description</code>	<code>text</code>	Yes	NULL		
<code>offline_sync_policy</code>	<code>varchar(255)</code>	No	'anytime'		
<code>created_at</code>	<code>timestamp</code>	Yes	NULL		
<code>updated_at</code>	<code>timestamp</code>	Yes	NULL		

enrollments

Student enrollment in courses per semester

Column	Type	Null	Default	Key	Notes
<code>enrollment_id</code>	<code>bigint(20) unsigned</code>	No		PK	auto-increment
<code>student_id</code>	<code>bigint(20) unsigned</code>	No		INDEX	
<code>course_id</code>	<code>bigint(20) unsigned</code>	No		INDEX	
<code>enrollment_date</code>	<code>date</code>	No			
<code>status</code>	<code>varchar(255)</code>	No	'active'		

Column	Type	Null	Default	Key	Notes
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id, course_id -> courses.course_id

exams

Exam timetable

Column	Type	Null	Default	Key	Notes
exam_id	bigint(20) unsigned	No		PK	auto-increment
course_id	bigint(20) unsigned	No		INDEX	
exam_name	varchar(255)	No			
exam_date	datetime	No			
room	varchar(255)	Yes	NULL		
class_group	varchar(255)	Yes	NULL		
max_score	int(11)	No	100		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: course_id -> courses.course_id

failed_jobs

Failed jobs

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
uuid	varchar(255)	No		UNIQUE	
connection	text	No			
queue	text	No			
payload	longtext	No			
exception	longtext	No			
failed_at	timestamp	No	current_timestamp()		

grade_entries

Per-student scores for each assessment event

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
grade_event_id	bigint(20) unsigned	No		INDEX	

Column	Type	Null	Default	Key	Notes
student_id	bigint(20) unsigned	No		INDEX	
score	decimal(5,2)	Yes	NULL		
notes	text	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id, grade_event_id -> grade_events.id

grade_events

Assessment events (exam / quiz / oral) with date, time and maximum score

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
teacher_id	bigint(20) unsigned	No		INDEX	
course_id	bigint(20) unsigned	Yes	NULL	INDEX	
program_id	bigint(20) unsigned	Yes	NULL		
year_level	tinyint(4)	Yes	NULL		
type	enum('exam','quiz','oral')	No		INDEX	
title	varchar(255)	No			
max_score	decimal(5,2)	No	100.00		
notes	text	Yes	NULL		
date	date	No			
time	varchar(255)	Yes	NULL		
duration	varchar(255)	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: teacher_id -> teachers.teacher_id, course_id -> courses.course_id

grade_report_requests

Head-of-department requests for grade reports from teachers

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
boss_user_id	bigint(20) unsigned	No			
teacher_user_id	bigint(20) unsigned	No			
course_id	bigint(20) unsigned	No		INDEX	
status	varchar(255)	No	'pending'		
notes	text	Yes	NULL		

Column	Type	Null	Default	Key	Notes
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: course_id -> courses.course_id

grades

Exam grades (legacy system)

Column	Type	Null	Default	Key	Notes
grade_id	bigint(20) unsigned	No		PK	auto-increment
student_id	bigint(20) unsigned	No		INDEX	
exam_id	bigint(20) unsigned	No		INDEX	
score	decimal(5,2)	No			
remarks	text	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id, exam_id -> exams.exam_id

group_user

Group members

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
group_id	bigint(20) unsigned	No		INDEX	
user_id	bigint(20) unsigned	No		INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: user_id -> users.user_id, group_id -> groups.id

groups

Chat groups

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
name	varchar(255)	No			
image	varchar(255)	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

head_schedule_entries

Timetable entries created by the head of department

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

heads

Heads of department and their department link

Column	Type	Null	Default	Key	Notes
head_id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
department_id	bigint(20) unsigned	No		INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: department_id -> departments.department_id, user_id -> users.user_id

job_batches

Job batches

Column	Type	Null	Default	Key	Notes
id	varchar(255)	No		PK	
name	varchar(255)	No			
total_jobs	int(11)	No			
pending_jobs	int(11)	No			
failed_jobs	int(11)	No			
failed_job_ids	longtext	No			
options	mediumtext	Yes	NULL		
cancelled_at	int(11)	Yes	NULL		
created_at	int(11)	No			
finished_at	int(11)	Yes	NULL		

jobs

Queue jobs

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment

Column	Type	Null	Default	Key	Notes
queue	varchar(255)	No		INDEX	
payload	longtext	No			
attempts	tinyint(3) unsigned	No			
reserved_at	int(10) unsigned	Yes	NULL		
available_at	int(10) unsigned	No			
created_at	int(10) unsigned	No			

leave_requests

Leave requests (student -> parent -> head -> affairs)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
student_id	bigint(20) unsigned	Yes	NULL	INDEX	
teacher_id	bigint(20) unsigned	Yes	NULL	INDEX	
type	enum('full_day', 'hourly')	No			
leave_category	enum('hourly', 'daily')	No	'daily'		
date	date	No			
reason	text	No			
attachment	varchar(255)	Yes	NULL		
status	varchar(50)	No	'pending_parent'	INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: teacher_id -> teachers.teacher_id, student_id -> users.user_id

lessons

Lectures / sessions (file, video, teacher, course); one attendance session is created per lesson

Column	Type	Null	Default	Key	Notes
lesson_id	bigint(20) unsigned	No		PK	auto-increment
course_id	bigint(20) unsigned	No		INDEX	
title	varchar(255)	No			
description	text	Yes	NULL		
file_path	varchar(255)	Yes	NULL		
file_name	varchar(255)	Yes	NULL		
file_type	varchar(255)	Yes	NULL		
content_url	varchar(255)	Yes	NULL		
type	varchar(255)	Yes	NULL		
file_size	varchar(255)	Yes	NULL		

Column	Type	Null	Default	Key	Notes
duration	varchar(255)	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		
teacher_id	bigint(20) unsigned	Yes	NULL	INDEX	
department_id	bigint(20) unsigned	Yes	NULL	INDEX	

Foreign keys: course_id -> courses.course_id, teacher_id -> teachers.teacher_id, department_id -> departments.department_id

messages

Chat messages (attachments, reply, forward, delete, disappearing, delivery/read)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
sender_id	bigint(20) unsigned	No		INDEX	
receiver_id	bigint(20) unsigned	Yes	NULL	INDEX	
group_id	bigint(20) unsigned	Yes	NULL		
course_id	bigint(20) unsigned	Yes	NULL		
message	text	Yes	NULL		
reply_to_message_id	bigint(20) unsigned	Yes	NULL		
attachment	varchar(255)	Yes	NULL		
is_read	tinyint(1)	No	0		
is_delivered	tinyint(1)	No	0		
deleted_for_sender	tinyint(1)	No	0		
deleted_for_receiver	tinyint(1)	No	0		
deleted_for_everyone	tinyint(1)	No	0		
expires_at	timestamp	Yes	NULL		
disappears_after	int(11)	Yes	NULL		
is_forwarded	tinyint(1)	No	0		
created_at	timestamp	Yes	NULL	INDEX	
updated_at	timestamp	Yes	NULL		

Foreign keys: receiver_id -> users.user_id, sender_id -> users.user_id

migrations

Column	Type	Null	Default	Key	Notes
id	int(10) unsigned	No		PK	auto-increment
migration	varchar(255)	No			
batch	int(11)	No			

notifications

In-system notifications (type, category, related id)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
sender_id	bigint(20) unsigned	Yes	NULL	INDEX	
title	varchar(255)	No			
message	text	No			
type	varchar(255)	No		INDEX	
related_id	bigint(20) unsigned	Yes	NULL		
category	enum('academic','administrative','chat')	Yes	'administrative'		
is_read	tinyint(1)	No	0	INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: user_id -> users.user_id, sender_id -> users.user_id

otp_codes

Email verification codes (current)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
email	varchar(255)	No		INDEX	
code	varchar(6)	No			
expires_at	timestamp	No	current_timestamp()		
used	tinyint(1)	No	0		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

otps

Verification codes (legacy)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
email	varchar(255)	No		INDEX	
token	varchar(255)	No			
expires_at	timestamp	No	current_timestamp()		
created_at	timestamp	Yes	NULL		

Column	Type	Null	Default	Key	Notes
updated_at	timestamp	Yes	NULL		

parent_meeting_requests

Meeting requests from parents

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
parent_user_id	bigint(20) unsigned	No		INDEX	
student_id	bigint(20) unsigned	Yes	NULL	INDEX	
target_role	varchar(255)	Yes	'affairs'		
department_id	bigint(20) unsigned	Yes	NULL		
target_user_id	bigint(20) unsigned	Yes	NULL		
subject	varchar(255)	No			
reason	text	No			
preferred_date	date	Yes	NULL		
status	enum('pending', 'approved', 'rejected', 'completed')	No	'pending'		
admin_response	text	Yes	NULL		
scheduled_at	datetime	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id, parent_user_id -> users.user_id

parent_students

Parent-to-student links (many-to-many)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
parent_id	bigint(20) unsigned	No		INDEX	
student_id	bigint(20) unsigned	No		INDEX	
relationship	enum('father', 'mother', 'guardian')	No	'father'		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> users.user_id, parent_id -> users.user_id

parent_summons

Parent summons (manual, or automatic after 10 absence days)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
sender_user_id	bigint(20) unsigned	No		INDEX	
student_id	bigint(20) unsigned	No		INDEX	
parent_user_id	bigint(20) unsigned	Yes	NULL	INDEX	
subject	varchar(255)	Yes	NULL		
reason	text	Yes	NULL		
date	varchar(255)	Yes	NULL		
time	varchar(255)	Yes	NULL		
reason_title	varchar(255)	No			
details	text	No			
summon_date	date	Yes	NULL		
status	enum('pending_hod','pending_affairs','sent','acknowledged','attended','cancelled','approved','rejected','completed')	No	'sent'	INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: sender_user_id -> users.user_id, student_id -> students.student_id

parents

Parent profile

Column	Type	Null	Default	Key	Notes
parent_id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		
telegram_id	varchar(255)	Yes	NULL		

Foreign keys: user_id -> users.user_id

performance_reports

Performance reports

Column	Type	Null	Default	Key	Notes
report_id	bigint(20) unsigned	No		PK	auto-increment
report_request_id	bigint(20) unsigned	Yes	NULL		
student_id	bigint(20) unsigned	No		INDEX	
report_type	enum('academic','behavioral')	No	'academic'		
attendance_rate	decimal(5,2)	No			
average_grade	decimal(5,2)	No			
recommendations	text	Yes	NULL		
generated_at	timestamp	No	current_timestamp()		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id

personal_access_tokens

Sanctum tokens

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
tokenable_type	varchar(255)	No		INDEX	
tokenable_id	bigint(20) unsigned	No			
name	text	No			
token	varchar(64)	No		UNIQUE	
abilities	text	Yes	NULL		
last_used_at	timestamp	Yes	NULL		
expires_at	timestamp	Yes	NULL	INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

photo_change_requests

Requests to change the student reference photo

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
old_photo	varchar(255)	Yes	NULL		
new_photo	varchar(255)	No			
status	enum('pending','approved','rejected')	No	'pending'		
reviewed_by	bigint(20) unsigned	Yes	NULL		
created_at	timestamp	Yes	NULL		

Column	Type	Null	Default	Key	Notes
updated_at	timestamp	Yes	NULL		

Foreign keys: user_id -> users.user_id

programs

Programs / specializations belonging to departments

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
name	varchar(255)	No			
description	text	Yes	NULL		
year	varchar(255)	Yes	NULL		
semester	varchar(255)	Yes	NULL		
department_id	bigint(20) unsigned	Yes	NULL	INDEX	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: department_id -> departments.department_id

quiz_options

Quiz question options

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
question_id	bigint(20) unsigned	No		INDEX	
option_text	varchar(255)	No			
is_correct	tinyint(1)	No	0		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: question_id -> quiz_questions.id

quiz_questions

Quiz questions

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
quiz_id	bigint(20) unsigned	No		INDEX	
question_text	text	No			
type	enum('mcq','text')	No	'mcq'		

Column	Type	Null	Default	Key	Notes
marks	smallint(5) unsigned	No	1		
order_num	smallint(5) unsigned	No	0		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: quiz_id -> quizzes.id

quizzes

Online quizzes

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
teacher_id	bigint(20) unsigned	No		INDEX	
course_id	bigint(20) unsigned	No		INDEX	
title	varchar(255)	No			
description	text	Yes	NULL		
duration_minutes	smallint(5) unsigned	No	60		
total_marks	smallint(5) unsigned	No	100		
start_at	datetime	Yes	NULL		
end_at	datetime	Yes	NULL		
is_published	tinyint(1)	No	0		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: teacher_id -> teachers.teacher_id, course_id -> courses.course_id

report_requests

Behavioral report requests (from head or parent) and head notes

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
head_id	bigint(20) unsigned	No		INDEX	
teacher_id	bigint(20) unsigned	Yes	NULL	INDEX	
student_id	bigint(20) unsigned	No		INDEX	
course_id	bigint(20) unsigned	Yes	NULL	INDEX	
year	tinyint(4)	Yes	NULL		
report_type	enum('academic','behavioral')	No			
notes	text	Yes	NULL		
hod_notes	text	Yes	NULL		

Column	Type	Null	Default	Key	Notes
status	enum('pending', 'completed')	No	'pending'		
sent_to_parent	tinyint(1)	No	0		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: course_id -> courses.course_id, teacher_id -> teachers.teacher_id, student_id -> students.student_id, head_id -> users.user_id

resources

Course files and references

Column	Type	Null	Default	Key	Notes
resource_id	bigint(20) unsigned	No		PK	auto-increment
course_id	bigint(20) unsigned	No		INDEX	
resource_name	varchar(255)	No			
file_path	varchar(255)	No			
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: course_id -> courses.course_id

roles

The six role definitions

Column	Type	Null	Default	Key	Notes
role_id	bigint(20) unsigned	No		PK	auto-increment
name	varchar(255)	No		UNIQUE	
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

schedules

Weekly class timetable (day, time, room, class group)

Column	Type	Null	Default	Key	Notes
schedule_id	bigint(20) unsigned	No		PK	auto-increment
course_id	bigint(20) unsigned	No		INDEX	
teacher_id	bigint(20) unsigned	No		INDEX	
class_group	varchar(255)	Yes	NULL		
day	varchar(255)	No		INDEX	

Column	Type	Null	Default	Key	Notes
start_time	time	No			
end_time	time	No			
room	varchar(255)	No			
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: course_id -> courses.course_id, teacher_id -> users.user_id

semesters

Academic semesters and the active-semester flag

Column	Type	Null	Default	Key	Notes
semester_id	bigint(20) unsigned	No		PK	auto-increment
name	varchar(255)	No			
start_date	date	No			
end_date	date	No			
is_active	tinyint(1)	No	0		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

session

Legacy sessions table

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

student_requests

Student service requests (appeal, documents, make-up exam, device unlock)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
student_id	bigint(20) unsigned	No		INDEX	
type	varchar(255)	No			
details	text	No			
status	varchar(255)	No	'pending_affairs'		
affairs_decision	enum('approved', 'rejected')	Yes	NULL		
hod_decision	enum('approved', 'rejected')	Yes	NULL		

Column	Type	Null	Default	Key	Notes
admin_decision	enum('approved','rejected')	Yes	NULL		
affairs_notes	text	Yes	NULL		
hod_notes	text	Yes	NULL		
admin_notes	text	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id

student_warnings

Automatic absence warnings (first / second / final)

Column	Type	Null	Default	Key	Notes
warning_id	bigint(20) unsigned	No		PK	auto-increment
student_id	bigint(20) unsigned	No		INDEX	
warning_level	enum('first','second','final')	No			
absence_days	int(10) unsigned	No			
message	text	No			
is_read	tinyint(1)	No	0		
action_data	longtext	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: student_id -> students.student_id

students

Student profile: code, level, program, device binding, reference face embedding

Column	Type	Null	Default	Key	Notes
student_id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
student_code	varchar(255)	No		UNIQUE	
device_id	varchar(255)	Yes	NULL		معرف الجهاز الوحيد المسموح له بتسجيل الحضور
web_device_token	varchar(255)	Yes	NULL		معرف الجهاز الموثوق به للويب للتحقق من الأجهزة الجديدة ومطابقة الوجه
is_device_locked	tinyint(1)	No	0		الجهاز مقفل ولا يمكن تغييره إلا من الأدمن = true
level	varchar(255)	Yes	NULL		

Column	Type	Null	Default	Key	Notes
face_embedding	longtext	Yes	NULL		
reference_photo	varchar(255)	Yes	NULL		
requires_face_reset	tinyint(1)	No	0		
birth_date	date	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		
program_id	bigint(20) unsigned	Yes	NULL	INDEX	

Foreign keys: user_id -> users.user_id, program_id -> programs.id

system_settings

System settings (theme)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
key	varchar(255)	No		UNIQUE	
value	text	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

teacher_dashboard_stats_view

View: teacher dashboard statistics

Column	Type	Null	Default	Key	Notes
teacher_id	bigint(20) unsigned	No	0		
courses_count	bigint(21)	Yes	NULL		
active_assignments_count	bigint(21)	Yes	NULL		

teachers

Teacher profile, plus advisor fields (advisor_branch / advisor_year)

Column	Type	Null	Default	Key	Notes
teacher_id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
specialization	varchar(255)	No			
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Column	Type	Null	Default	Key	Notes
advisor_branch	varchar(255)	Yes	NULL		
advisor_year	varchar(255)	Yes	NULL		
advisor_section	varchar(255)	Yes	NULL		

Foreign keys: user_id -> users.user_id

university_ids

Pre-registered university IDs created by Affairs before a student registers

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
university_id	varchar(255)	No		UNIQUE	
full_name	varchar(255)	No			
first_name	varchar(255)	Yes	NULL		
last_name	varchar(255)	Yes	NULL		
date_of_birth	date	Yes	NULL		
phone	varchar(20)	Yes	NULL		
photo	varchar(255)	Yes	NULL		
role	enum('student', 'parent')	No	'student'		
telegram_chat_id	varchar(255)	Yes	NULL		
is_used	tinyint(1)	No	0		
created_by	bigint(20) unsigned	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

user_activities

User activity log (audit)

Column	Type	Null	Default	Key	Notes
id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	Yes	NULL	INDEX	
user_name	varchar(255)	Yes	NULL		
role_name	varchar(255)	Yes	NULL	INDEX	
action	varchar(255)	No		INDEX	
description	text	Yes	NULL		
ip_address	varchar(255)	Yes	NULL		
user_agent	text	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

user_activity

Legacy activity log

Column	Type	Null	Default	Key	Notes
activity_id	bigint(20) unsigned	No		PK	auto-increment
user_id	bigint(20) unsigned	No		INDEX	
activity_type	varchar(255)	No			
activity_time	timestamp	No	current_timestamp()		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: user_id -> users.user_id

users

Core accounts for all roles (login, role, status, session, account lockout, FCM token, Telegram id)

Column	Type	Null	Default	Key	Notes
user_id	bigint(20) unsigned	No		PK	auto-increment
role_id	bigint(20) unsigned	No		INDEX	
full_name	varchar(255)	No			
first_name	varchar(255)	Yes	NULL		
last_name	varchar(255)	Yes	NULL		
username	varchar(255)	No		UNIQUE	
email	varchar(255)	Yes	NULL	UNIQUE	
avatar	varchar(255)	Yes	NULL		
password	varchar(255)	No			
phone	varchar(255)	Yes	NULL		
telegram_id	varchar(255)	Yes	NULL		
telegram_chat_id	varchar(255)	Yes	NULL		
university_id	varchar(255)	Yes	NULL	UNIQUE	
department	varchar(255)	Yes	NULL		
branch	varchar(255)	Yes	NULL		
children_ids	longtext	Yes	NULL		
gender	enum('ذكر', 'أنثى')	Yes	NULL		
birth_date	date	Yes	NULL		
academic_year	varchar(255)	Yes	NULL		
status	enum('active', 'inactive')	No	'active'	INDEX	
device_token	varchar(255)	Yes	NULL		
notifications_muted	tinyint(1)	No	0		

Column	Type	Null	Default	Key	Notes
last_login	timestamp	Yes	NULL		
failed_login_attempts	int(10) unsigned	No	0		
locked_until	timestamp	Yes	NULL		
remember_token	varchar(100)	Yes	NULL		
current_session_id	varchar(255)	Yes	NULL		
current_token_id	bigint(20) unsigned	Yes	NULL		
session_last_active_at	timestamp	Yes	NULL		
created_at	timestamp	Yes	NULL		
updated_at	timestamp	Yes	NULL		

Foreign keys: role_id -> roles.role_id

API Reference

Auto-generated from `php artisan route:list` (330 endpoints under `/api`). **Base:** `<SERVER>/api` (local example: `http://127.0.0.1:8000/api`).

General usage

- **Authentication:** Laravel Sanctum. After `POST /api/login` a `token` is returned and must be sent on every request: `Authorization: Bearer <token>` with `Accept: application/json`.
- **Single session:** logging in from a new device is refused while another session is active (`423`). A token that was invalidated receives `401` with `error_code = LOGGED_IN_ELSEWHERE`.
- **Roles:** `student`, `teacher`, `head` (head of department), `admin`, `parent`, `affairs`. A disallowed role gets `403`.
- **Rate limits:** a global `throttle:api` (240 requests/minute per user). Login and OTP routes have stricter limits (shown in the table). Exceeding returns `429`.
- **Response shape:** JSON, usually `{ "success": true|false, "message": "...", "data": ... }`. Errors: `422` validation, `403` permission, `404` not found, `423` account locked or session occupied.
- **Encoding:** UTF-8; Arabic text is returned unescaped.
- **Note:** handler names (`Controller@method`) are descriptive and are the starting point for reading each endpoint's logic in code.

Index

- Public (no authentication): **12**
- Shared (authentication only): **28**
- Student (`/api/student/*`): **35**
- Teacher (`/api/teacher/*`): **59**
- Head of department (`/api/department-head/*`): **51**
- Admin (`/api/admin/*`): **42**
- Parent (`/api/parent/*`): **38**
- Affairs (`/api/affairs/*`): **65**

Public (no authentication)

Method	Path	Handler	Access
GET	<code>/api/file/{path}</code>	<code>Closure</code>	Public
POST	<code>/api/forgot-password</code>	<code>Api\AuthController@forgotPassword</code>	Public (throttle:otp-send)
POST	<code>/api/login</code>	<code>Api\AuthController@login</code>	Public (throttle:login)
POST	<code>/api/login-otp/send</code>	<code>Api\AuthController@sendLoginOtp</code>	Public (throttle:login-otp)
POST	<code>/api/login-otp/verify</code>	<code>Api\AuthController@verifyLoginOtp</code>	Public (throttle:otp-verify)
POST	<code>/api/register</code>	<code>Api\AuthController@register</code>	Public (throttle:otp-send)
POST	<code>/api/request-device-reset</code>	<code>Api\AuthController@requestDeviceReset</code>	Public (throttle:login)

Method	Path	Handler	Access
POST	/api/resend-otp	Api\AuthController@resendOtp	Public (throttle:otp-send)
POST	/api/reset-password	Api\AuthController@resetPassword	Public (throttle:otp-verify)
GET	/api/system/settings	Closure	Public
POST	/api/telegram/webhook	Api\TelegramWebhookController@handle	Public
POST	/api/verify-otp	Api\AuthController@verifyOtp	Public (throttle:otp-verify)

Shared (authentication only)

Method	Path	Handler	Access
POST	/api/ai/chat	Api\AiAssistantController@chat	Auth (any role)
GET	/api/attendance/{attendanceId}/face	Api\FaceImageController@show	Auth (any role)
POST	/api/broadcasting/auth	Closure	Auth (any role)
GET	/api/contacts	ChatController@getContacts	Auth (any role)
POST	/api/groups	ChatController@createGroup	Auth (any role)
GET	/api/groups/{groupId}/messages	ChatController@getGroupMessages	Auth (any role)
POST	/api/groups/{groupId}/messages	ChatController@sendGroupMessage	Auth (any role)
POST	/api/logout	Api\AuthController@logout	Auth (any role)
POST	/api/messages/forward	ChatController@forwardMessage	Auth (any role)
GET	/api/messages/unread-count	ChatController@getUnreadCount	Auth (any role)
GET	/api/messages/{id}/download	ChatController@downloadAttachment	Auth (any role)
DELETE	/api/messages/{messageId}	ChatController@deleteMessage	Auth (any role)
PUT	/api/messages/{messageId}/edit	ChatController@editMessage	Auth (any role)
GET	/api/messages/{otherUserId}	ChatController@getMessages	Auth (any role)
PUT	/api/messages/{otherUserId}/mark-read	ChatController@markAsRead	Auth (any role)
GET	/api/messages/{otherUserId}/search	ChatController@searchMessages	Auth (any role)
GET	/api/parent/info/{user_id}	Closure	Auth (any role)

Method	Path	Handler	Access
POST	/api/parent/link-student	Closure	Auth (any role)
POST	/api/profile/avatar	Api\AuthController@updateAvatar	Auth (any role)
POST	/api/profile/confirm-change-email	Api\AuthController@confirmChangeEmail	Auth (any role)
POST	/api/profile/request-change-email	Api\AuthController@requestChangeEmail	Auth (any role)
POST	/api/profile/send-otp	Api\AuthController@sendProfileOtp	Auth (any role)
POST	/api/profile/update	Api\AuthController@updateProfile	Auth (any role)
POST	/api/profile/verify-otp	Api\AuthController@verifyProfileOtp	Auth (any role)
POST	/api/send-message	ChatController@sendMessage	Auth (any role)
GET	/api/transcript/export-pdf	Api\AffairsController@exportCourseWeightsStudentPdf	Auth (any role)
POST	/api/user/fcm-token	Closure	Auth (any role)
GET	/api/user/profile	Closure	Auth (any role)

Student (/api/student/*)

Method	Path	Handler	Access
GET	POST	HEAD	/api/student/academic-card
GET	/api/student/academic-card/export-pdf	Api\StudentController@exportAcademicCardPdf	Auth + role: student
GET	/api/student/announcements	Api\AnnouncementController@getHomeAnnouncements	Auth + role: student
GET	/api/student/assignments	Api\StudentController@getMyAssignments	Auth + role: student
POST	/api/student/assignments/{id}/submit	Api\StudentController@submitAssignment	Auth + role: student
GET	/api/student/attendance	Api\StudentController@getMyAttendance	Auth + role: student
POST	/api/student/attendance/scan	Api\StudentController@scanAttendanceQr	Auth + role: student
POST	/api/student/attendance/{attendance_id}/excuse	Api\StudentController@submitAttendanceExcuse	Auth + role: student
GET	/api/student/courses	Api\StudentController@getMyCourses	Auth + role: student
GET	/api/student/courses/{courseId}/materials	Api\StudentController@getCourseMaterials	Auth + role: student
GET	/api/student/dashboard	Api\StudentController@getDashboardData	Auth + role: student

Method	Path	Handler	Access
GET	/api/student/grade-event/{id}	Api\StudentController@getGradeEventForStudent	Auth + role: student
GET	/api/student/grades	Api\StudentController@getMyGrades	Auth + role: student
GET	/api/student/info/{id}	Closure	Auth + role: student,parent
GET	/api/student/leave-requests	Api\StudentController@getMyAbsenceRequests	Auth + role: student
POST	/api/student/leave-requests	Api\StudentController@requestAbsence	Auth + role: student
GET	/api/student/leave-requests/{id}	Api\StudentController@getLeaveDetails	Auth + role: student
GET	/api/student/lectures	Api\StudentController@getMyLectures	Auth + role: student
GET	/api/student/my-exams	Api\StudentController@getMyExams	Auth + role: student
GET	/api/student/my-exams/excel	Api\StudentController@exportExamsExcel	Auth + role: student
GET	/api/student/my-exams/pdf	Api\StudentController@exportExamsPdf	Auth + role: student
GET	/api/student/my-schedule	Api\StudentController@getMySchedule	Auth + role: student
GET	/api/student/my-schedule/pdf	Api\StudentController@exportSchedulePdf	Auth + role: student
GET	/api/student/notifications	Api\StudentController@getNotifications	Auth + role: student
PUT	/api/student/notifications/read-all	Api\StudentController@markAllNotificationsAsRead	Auth + role: student
PUT	/api/student/notifications/{id}/read	Api\StudentController@markNotificationAsRead	Auth + role: student
POST	/api/student/photo-change-request	Api\StudentController@requestPhotoChange	Auth + role: student
GET	/api/student/photo-change-request/status	Api\StudentController@myPhotoChangeStatus	Auth + role: student
GET	/api/student/profile	Api\StudentController@getProfileData	Auth + role: student
POST	/api/student/profile/initialize-face	Api\StudentController@initializeFaceFromPhoto	Auth + role: student
POST	/api/student/profile/update	Api\StudentController@updateProfile	Auth + role: student
GET	/api/student/program-courses	Api\StudentController@getProgramCourses	Auth + role: student
GET	/api/student/services/requests	Api\StudentController@getMyRequests	Auth + role: student
POST	/api/student/services/requests	Api\StudentController@submitRequest	Auth + role: student
GET	/api/student/warnings	Api\StudentController@getMyWarnings	Auth + role: student

Teacher (/api/teacher/*)

Method	Path	Handler	Access
GET	/api/teacher/announcements	Api\TeacherController@getAnnouncements	Auth + role: teacher
POST	/api/teacher/announcements	Api\TeacherController@createAnnouncement	Auth + role: teacher

Method	Path	Handler	Access
GET	/api/teacher/assignments	Api\TeacherController@getAssignments	Auth + role: teacher
POST	/api/teacher/assignments	Api\TeacherController@createAssignment	Auth + role: teacher
DELETE	/api/teacher/assignments/{assignmentId}	Api\TeacherController@deleteAssignment	Auth + role: teacher
PUT	/api/teacher/assignments/{assignmentId}	Api\TeacherController@updateAssignment	Auth + role: teacher
GET	/api/teacher/assignments/{assignmentId}/submissions	Api\TeacherController@getAssignmentSubmissions	Auth + role: teacher
POST	/api/teacher/assignments/{submissionId}/grade	Api\TeacherController@gradeAssignment	Auth + role: teacher
POST	/api/teacher/attendance	Api\TeacherController@markAttendance	Auth + role: teacher
GET	/api/teacher/attendance/advisor-export	Api\TeacherController@advisorExportAttendance	Auth + role: teacher
GET	/api/teacher/attendance/export	Api\TeacherController@exportAttendance	Auth + role: teacher
GET	/api/teacher/attendance/export-pdf	Api\TeacherController@exportFilteredPdf	Auth + role: teacher
POST	/api/teacher/attendance/generate-qr	Api\TeacherController@generateQrSession	Auth + role: teacher
POST	/api/teacher/attendance/session/{sessionId}/end	Api\TeacherController@endSession	Auth + role: teacher
GET	/api/teacher/attendance/session/{sessionId}/list	Api\TeacherController@getSessionAttendance	Auth + role: teacher
POST	/api/teacher/attendance/session/{sessionId}/refresh-qr	Api\TeacherController@refreshQrToken	Auth + role: teacher
GET	/api/teacher/attendance/{courseId}	Api\TeacherController@getAttendance	Auth + role: teacher
GET	/api/teacher/courses	Api\TeacherController@myCourses	Auth + role: teacher
GET	/api/teacher/courses/{courseId}/students	Api\TeacherController@courseStudents	Auth + role: teacher
GET	/api/teacher/dashboard	Api\TeacherController@dashboard	Auth + role: teacher
GET	/api/teacher/educator-students	Api\TeacherController@getEducatorStudents	Auth + role: teacher
GET	/api/teacher/exams	Api\TeacherController@getExams	Auth + role: teacher
POST	/api/teacher/exams	Api\TeacherController@createExam	Auth + role: teacher
GET	/api/teacher/grade-report-requests/pending	Api\TeacherController@getPendingGradeReportRequests	Auth + role: teacher

Method	Path	Handler	Access
POST	/api/teacher/grade-report-requests/{id}/complete	Api\TeacherController@completeGradeReport	Auth + role: teacher
POST	/api/teacher/grades	Api\TeacherController@enterGrades	Auth + role: teacher
GET	/api/teacher/grades/events	Api\TeacherController@getGradeEvents	Auth + role: teacher
POST	/api/teacher/grades/events	Api\TeacherController@createGradeEvent	Auth + role: teacher
DELETE	/api/teacher/grades/events/{id}	Api\TeacherController@deleteGradeEvent	Auth + role: teacher
GET	/api/teacher/grades/events/{id}/entries	Api\TeacherController@getGradeEntries	Auth + role: teacher
POST	/api/teacher/grades/events/{id}/entries	Api\TeacherController@saveGradeEntries	Auth + role: teacher
GET	/api/teacher/grades/program-students	Api\TeacherController@getProgramStudents	Auth + role: teacher
GET	/api/teacher/grades/programs	Api\TeacherController@getTeacherPrograms	Auth + role: teacher
GET	/api/teacher/grades/{courseId}	Api\TeacherController@getGrades	Auth + role: teacher
GET	/api/teacher/lessons	Api\TeacherController@getLessons	Auth + role: teacher
POST	/api/teacher/lessons	Api\TeacherController@createLesson	Auth + role: teacher
DELETE	/api/teacher/lessons/{lessonId}	Api\TeacherController@deleteLesson	Auth + role: teacher
POST	/api/teacher/lessons/{lessonId}	Api\TeacherController@updateLesson	Auth + role: teacher
GET	/api/teacher/messages	Api\TeacherController@getMessages	Auth + role: teacher
POST	/api/teacher/messages	Api\TeacherController@sendMessage	Auth + role: teacher
GET	/api/teacher/notifications	Api\TeacherController@getNotifications	Auth + role: teacher
PUT	/api/teacher/notifications/read-all	Api\TeacherController@markAllNotificationsRead	Auth + role: teacher
PUT	/api/teacher/notifications/{notificationId}/read	Api\TeacherController@markNotificationRead	Auth + role: teacher
GET	/api/teacher/parent-summons	Api\ParentMeetingController@listSummons	Auth + role: teacher
GET	/api/teacher/parent-summons-history	Api\TeacherController@getTeacherSummonsHistory	Auth + role: teacher
POST	/api/teacher/parent-summons/request	Api\TeacherController@sendParentSummon	Auth + role: teacher

Method	Path	Handler	Access
POST	/api/teacher/parent-summons/send	Api\TeacherController@sendParentSummon	Auth + role: teacher
GET	/api/teacher/profile	Api\TeacherController@getTeacherProfile	Auth + role: teacher
PUT	/api/teacher/profile	Api\TeacherController@updateTeacherProfile	Auth + role: teacher
POST	/api/teacher/profile/avatar	Api\TeacherController@updateAvatar	Auth + role: teacher
GET	/api/teacher/programs	Api\TeacherController@myDepartmentPrograms	Auth + role: teacher
GET	/api/teacher/report-requests	Api\TeacherController@getReportRequests	Auth + role: teacher
GET	/api/teacher/report-requests/{id}/stats	Api\TeacherController@getStudentAcademicStats	Auth + role: teacher
POST	/api/teacher/report-requests/{id}/submit	Api\TeacherController@submitEvaluation	Auth + role: teacher
GET	/api/teacher/schedule	Api\TeacherController@getSchedule	Auth + role: teacher
POST	/api/teacher/students/{studentId}/reset-face	Api\TeacherController@resetStudentFace	Auth + role: teacher
GET	/api/teacher/submissions	Api\TeacherController@getSubmissions	Auth + role: teacher

Head of department (/api/department-head/*)

Method	Path	Handler	Access
GET	/api/department-head/all-exams	Api\DepartmentHeadController@getAllExams	Auth + role: head
GET	/api/department-head/all-schedule	Api\DepartmentHeadController@getAllSchedule	Auth + role: head
GET	/api/department-head/announcements	Api\DepartmentHeadController@getAnnouncements	Auth + role: head
POST	/api/department-head/announcements	Api\DepartmentHeadController@createAnnouncement	Auth + role: head
DELETE	/api/department-head/announcements/{id}	Api\DepartmentHeadController@deleteAnnouncement	Auth + role: head
POST	/api/department-head/announcements/{id}	Api\DepartmentHeadController@updateAnnouncement	Auth + role: head
GET	/api/department-head/appointments/meetings	Api\HODController@getMeetingRequests	Auth + role: head
POST	/api/department-head/appointments/meetings/{id}/respond	Api\HODController@respondToMeetingRequest	Auth + role: head

Meth od	Path	Handler	Access
PUT	/api/department-head/appointments/meetings/{id}/respond	Api\HODController@respondToMeetingRequest	Auth + role: head
GET	/api/department-head/appointments/metadata	Api\HODController@getAppointmentsMetadata	Auth + role: head
GET	/api/department-head/appointments/summons	Api\HODController@getSummons	Auth + role: head
POST	/api/department-head/appointments/summons	Api\HODController@storeSummon	Auth + role: head
POST	/api/department-head/appointments/summons/{id}/forward	Api\HODController@forwardSummonToAffairs	Auth + role: head
GET	/api/department-head/courses	Api\DepartmentHeadController@getCourses	Auth + role: head
GET	/api/department-head/courses/{id}/students	Api\DepartmentHeadController@getStudentsByCourse	Auth + role: head
GET	/api/department-head/courses/{id}/teachers	Api\DepartmentHeadController@getTeachersByCourse	Auth + role: head
GET	/api/department-head/dashboard	Api\DepartmentHeadController@dashboard	Auth + role: head
GET	/api/department-head/grade-report-requests	Api\DepartmentHeadController@getGradeReports	Auth + role: head
POST	/api/department-head/grade-report-requests	Api\DepartmentHeadController@requestGradeReport	Auth + role: head
GET	/api/department-head/grade-report-requests/{courseId}/entries	Api\DepartmentHeadController@getCourseGradeEntries	Auth + role: head
POST	/api/department-head/grade-report-requests/{courseId}/remind-teacher	Api\DepartmentHeadController@remindTeacher	Auth + role: head
GET	/api/department-head/leave-requests	Api\DepartmentHeadController@getLeaveRequests	Auth + role: head
PUT	/api/department-head/leave-requests/{id}/respond	Api\DepartmentHeadController@respondLeaveRequest	Auth + role: head
GET	/api/department-head/metadata	Api\DepartmentHeadController@getMetadata	Auth + role: head
GET	/api/department-head/notifications	Api\DepartmentHeadController@getNotifications	Auth + role: head
PUT	/api/department-head/notifications/read-all	Api\DepartmentHeadController@markAllNotificationsRead	Auth + role: head
POST	/api/department-head/notifications/send	Api\DepartmentHeadController@sendNotification	Auth + role: head
PUT	/api/department-head/notifications/{id}/read	Api\DepartmentHeadController@markNotificationRead	Auth + role: head
GET	/api/department-head/parent-meetings	Api\ParentMeetingController@listMeetingRequests	Auth + role: head
PUT	/api/department-head/parent-meetings/{id}/respond	Api\ParentMeetingController@respondToMeetingRequest	Auth + role: head

Meth od	Path	Handler	Access
GET	/api/department-head/parent-summons	Api\ParentMeetingController@listSummons	Auth + role: head
POST	/api/department-head/parent-summons	Api\ParentMeetingController@sendSummon	Auth + role: head
GET	/api/department-head/profile	Api\DepartmentHeadController@getProfile	Auth + role: head
GET	/api/department-head/programs-schedule	Api\DepartmentHeadController@getProgramsS chedule	Auth + role: head
GET	/api/department-head/report-requests	Api\DepartmentHeadController@getReportReq uests	Auth + role: head
POST	/api/department-head/report-requests	Api\DepartmentHeadController@createReport Request	Auth + role: head
DELET E	/api/department-head/report-requests/{id}	Api\DepartmentHeadController@deleteReport Request	Auth + role: head
POST	/api/department-head/report-requests/{id}/hod- notes	Api\DepartmentHeadController@updateHodNot es	Auth + role: head
POST	/api/department-head/report-requests/{id}/send- to-parent	Api\DepartmentHeadController@sendReportTo Parent	Auth + role: head
GET	/api/department-head/schedule	Api\DepartmentHeadController@getSchedule	Auth + role: head
POST	/api/department-head/schedule	Api\DepartmentHeadController@createSchedu le	Auth + role: head
PUT	/api/department-head/schedule/{id}	Api\DepartmentHeadController@updateSchedu le	Auth + role: head
GET	/api/department-head/student-service-requests	Api\DepartmentHeadController@getStudentSe rvceRequests	Auth + role: head
PUT	/api/department-head/student-service- requests/{id}/respond	Api\DepartmentHeadController@respondStude ntServiceRequest	Auth + role: head
GET	/api/department-head/teachers	Api\DepartmentHeadController@getTeachers	Auth + role: head
POST	/api/department-head/users/parent	Api\DepartmentHeadController@createParen t	Auth + role: head
GET	/api/department-head/users/parents	Api\DepartmentHeadController@getParents	Auth + role: head
POST	/api/department-head/users/student	Api\DepartmentHeadController@createStuden t	Auth + role: head
GET	/api/department-head/users/students	Api\DepartmentHeadController@getStudents	Auth + role: head
POST	/api/department-head/users/trainer	Api\DepartmentHeadController@createTraine r	Auth + role: head
GET	/api/department-head/users/trainers	Api\DepartmentHeadController@getTrainers	Auth + role: head

Admin (/api/admin/*)

Method	Path	Handler	Access
POST	/api/admin/accounts/{id}/approve	Api\AdminController@approveAccount	Auth + role: admin
POST	/api/admin/accounts/{id}/reject	Api\AdminController@rejectAccount	Auth + role: admin
POST	/api/admin/announcements	Api\AdminController@createAnnouncement	Auth + role: admin
DELETE	/api/admin/announcements/{id}	Api\AdminController@deleteAnnouncement	Auth + role: admin
POST	/api/admin/announcements/{id}	Api\AdminController@updateAnnouncement	Auth + role: admin
GET	/api/admin/assign-hod	Api\AdminController@getAssignHodData	Auth + role: admin
POST	/api/admin/assign-hod	Api\AdminController@assignHodExisting	Auth + role: admin
POST	/api/admin/assign-hod/new	Api\AdminController@assignHodNew	Auth + role: admin
POST	/api/admin/broadcast	Api\AdminController@sendBroadcast	Auth + role: admin
GET	/api/admin/courses	Api\AdminController@getCourses	Auth + role: admin
POST	/api/admin/courses	Api\AdminController@createCourse	Auth + role: admin
DELETE	/api/admin/courses/{id}	Api\AdminController@deleteCourse	Auth + role: admin
GET	/api/admin/courses/{id}	Api\AdminController@getCourse	Auth + role: admin
PUT	/api/admin/courses/{id}	Api\AdminController@updateCourse	Auth + role: admin
GET	/api/admin/dashboard	Api\AdminController@dashboard	Auth + role: admin
GET	/api/admin/departments	Api\AdminController@getDepartments	Auth + role: admin
POST	/api/admin/departments	Api\AdminController@createDepartment	Auth + role: admin
POST	/api/admin/departments/assign-programs	Api\AdminController@assignProgramsToDepartment	Auth + role: admin
DELETE	/api/admin/departments/{id}	Api\AdminController@deleteDepartment	Auth + role: admin
PUT	/api/admin/departments/{id}	Api\AdminController@updateDepartment	Auth + role: admin
GET	/api/admin/parent-meetings	Api\ParentMeetingController@listMeetingRequests	Auth + role: admin
PUT	/api/admin/parent-meetings/{id}/respond	Api\ParentMeetingController@respondToMeetingRequest	Auth + role: admin

Method	Path	Handler	Access
GET	/api/admin/parent-summons	Api\ParentMeetingController@listSummons	Auth + role: admin
POST	/api/admin/parent-summons	Api\ParentMeetingController@sendSummon	Auth + role: admin
GET	/api/admin/pending-accounts	Api\AdminController@getPendingAccounts	Auth + role: admin
GET	/api/admin/reports/attendance	Api\AdminController@attendanceReport	Auth + role: admin
GET	/api/admin/reports/export/{id}	Api\AdminController@exportReportById	Auth + role: admin
GET	/api/admin/reports/grades	Api\AdminController@gradesReport	Auth + role: admin
GET	/api/admin/reports/log	Api\AdminController@getReportsLog	Auth + role: admin
GET	/api/admin/reports/students	Api\AdminController@studentsReport	Auth + role: admin
GET	/api/admin/reports/view/{id}	Api\AdminController@viewReportById	Auth + role: admin
GET	/api/admin/semesters	Api\AdminController@getSemesters	Auth + role: admin
POST	/api/admin/semesters	Api\AdminController@createSemester	Auth + role: admin
GET	/api/admin/semesters-subjects	Api\AdminController@getSemestersSubjects	Auth + role: admin
DELETE	/api/admin/semesters/{id}	Api\AdminController@deleteSemester	Auth + role: admin
PUT	/api/admin/semesters/{id}	Api\AdminController@updateSemester	Auth + role: admin
GET	/api/admin/student-services	Api\AdminController@getStudentServices	Auth + role: admin
POST	/api/admin/student-services/{id}/process	Api\AdminController@processStudentService	Auth + role: admin
GET	/api/admin/users	Api\AdminController@getUsers	Auth + role: admin
POST	/api/admin/users	Api\AdminController@createUser	Auth + role: admin
DELETE	/api/admin/users/{id}	Api\AdminController@deleteUser	Auth + role: admin
PUT	/api/admin/users/{id}	Api\AdminController@updateUser	Auth + role: admin

Parent (/api/parent/*)

Method	Path	Handler	Access
POST	/api/parent/add-student	Api\ParentController@linkStudent	Auth + role: parent
DELETE	/api/parent/affairs/notifications/{id}	NotificationController@deleteNotification	Auth + role: parent
GET	/api/parent/announcements	Api\ParentController@getAnnouncements	Auth + role: parent
DELETE	/api/parent/children/{id}	Api\ParentController@unlinkStudent	Auth + role: parent
GET	/api/parent/children/{id}/academic-card	Api\ParentController@getChildAcademicCard	Auth + role: parent
GET	/api/parent/children/{id}/academic-card/export-excel	Api\ParentController@exportChildAcademicCardExcel	Auth + role: parent
GET	/api/parent/children/{id}/academic-card/export-pdf	Api\ParentController@exportChildAcademicCardPdf	Auth + role: parent
GET	/api/parent/children/{id}/assignments	Api\ParentController@getChildAssignments	Auth + role: parent
GET	/api/parent/children/{id}/attendance	Api\ParentController@getChildAttendance	Auth + role: parent
GET	/api/parent/children/{id}/details	Api\ParentController@getChildDetails	Auth + role: parent
GET	/api/parent/children/{id}/grades	Api\ParentController@getChildGrades	Auth + role: parent
GET	/api/parent/children/{id}/schedule	Api\ParentController@getChildSchedule	Auth + role: parent
POST	/api/parent/children/{id}/unlink	Api\ParentController@unlinkStudent	Auth + role: parent
GET	/api/parent/children/{parent_id?}	Api\ParentController@getChildren	Auth + role: parent
GET	/api/parent/dashboard	Api\ParentController@dashboard	Auth + role: parent
DELETE	/api/parent/hod/notifications/{id}	NotificationController@deleteNotification	Auth + role: parent
GET	/api/parent/leave-requests	StudentParentController@getLeaveRequests	Auth + role: parent
POST	/api/parent/leave-requests/submit	StudentParentController@submitParentLeaveRequest	Auth + role: parent
POST	/api/parent/leave-requests/{id}/respond	StudentParentController@respondLeaveRequest	Auth + role: parent
GET	/api/parent/meeting-requests	Api\ParentController@getMyMeetingRequests	Auth + role: parent
GET	/api/parent/notifications	NotificationController@getNotifications	Auth + role: parent
DELETE	/api/parent/notifications/chat/{senderId}	NotificationController@deleteChatNotifications	Auth + role: parent

Method	Path	Handler	Access
PUT	/api/parent/notifications/read-all	NotificationController@markAllAsRead	Auth + role: parent
PUT	/api/parent/notifications/read-by-type	NotificationController@markByTypeAndRelatedId	Auth + role: parent
POST	/api/parent/notifications/toggle-mute	NotificationController@toggleMute	Auth + role: parent
DELETE	/api/parent/notifications/{id}	NotificationController@deleteNotification	Auth + role: parent
PUT	/api/parent/notifications/{id}/read	NotificationController@markAsRead	Auth + role: parent
GET	/api/parent/performance/{studentId}	StudentParentController@getFullPerformance	Auth + role: parent
GET	/api/parent/reports/history	Api\ParentController@getReportsHistory	Auth + role: parent
POST	/api/parent/request-meeting	Api\ParentController@requestMeeting	Auth + role: parent
POST	/api/parent/request-report	Api\ParentController@requestReport	Auth + role: parent
DELETE	/api/parent/student/notifications/{id}	NotificationController@deleteNotification	Auth + role: parent
GET	/api/parent/student/{studentId}/assignments	StudentParentController@getAssignments	Auth + role: parent
GET	/api/parent/summons	Api\ParentController@getMySummons	Auth + role: parent
POST	/api/parent/summons/{id}/respond	Api\ParentMeetingController@respondToSummon	Auth + role: parent
DELETE	/api/parent/teacher/notifications/{id}	NotificationController@deleteNotification	Auth + role: parent

Affairs (/api/affairs/*)

Method	Path	Handler	Access
GET	/api/affairs/academic-card	Api\AffairsController@getStudentAcademicCardForAffairs	Auth + role: affairs,admin
GET	/api/affairs/academic-card/export-pdf	Api\AffairsController@exportStudentAcademicCardPdf	Auth + role: affairs,admin
GET	/api/affairs/accounts	Api\AffairsController@listAccounts	Auth + role: affairs,admin
POST	/api/affairs/accounts/create	Api\AffairsController@createAccount	Auth + role: affairs,admin
DELETE	/api/affairs/accounts/{id}	Api\AffairsController@deleteAccount	Auth + role: affairs,admin

Method	Path	Handler	Access
POST	/api/affairs/accounts/{id}/toggle	Api\AffairsController@toggleAccountStatus	Auth + role: affairs,admin
POST	/api/affairs/accounts/{id}/update	Api\AffairsController@updateAccount	Auth + role: affairs,admin
POST	/api/affairs/accounts/{userId}/approve	Api\AffairsController@approveAccount	Auth + role: affairs,admin
POST	/api/affairs/accounts/{userId}/reject	Api\AffairsController@rejectAccount	Auth + role: affairs,admin
GET	/api/affairs/announcements	Api\AffairsController@listAnnouncements	Auth + role: affairs,admin
POST	/api/affairs/announcements	Api\AffairsController@createAnnouncement	Auth + role: affairs,admin
DELETE	/api/affairs/announcements/{id}	Api\AffairsController@deleteAnnouncement	Auth + role: affairs,admin
POST	/api/affairs/announcements/{id}	Api\AffairsController@updateAnnouncement	Auth + role: affairs,admin
GET	/api/affairs/appointments/meetings	Api\AffairsController@getMeetingRequests	Auth + role: affairs,admin
POST	/api/affairs/appointments/meetings/{id}/respond	Api\AffairsController@respondToMeetingRequest	Auth + role: affairs,admin
PUT	/api/affairs/appointments/meetings/{id}/respond	Api\AffairsController@respondToMeetingRequest	Auth + role: affairs,admin
GET	/api/affairs/appointments/metadata	Api\AffairsController@getAppointmentsMetadata	Auth + role: affairs,admin
GET	/api/affairs/appointments/summons	Api\AffairsController@getSummons	Auth + role: affairs,admin
POST	/api/affairs/appointments/summons	Api\AffairsController@storeSummon	Auth + role: affairs,admin
POST	/api/affairs/appointments/summons/{id}/issue	Api\AffairsController@issueParentSummon	Auth + role: affairs,admin
POST	/api/affairs/broadcasting/auth	Closure	Auth + role: affairs,admin
GET	/api/affairs/calendar	Api\AffairsController@listCalendarEvents	Auth + role: affairs,admin
POST	/api/affairs/calendar/events	Api\AffairsController@storeCalendarEvent	Auth + role: affairs,admin
POST	/api/affairs/calendar/events/delete/{id}	Api\AffairsController@deleteCalendarEvent	Auth + role: affairs,admin
POST	/api/affairs/calendar/events/update/{id}	Api\AffairsController@updateCalendarEvent	Auth + role: affairs,admin
GET	/api/affairs/course-weights/data	Api\AffairsController@getCourseWeightsData	Auth + role: affairs,admin
GET	/api/affairs/course-weights/export-cohort-pdf	Api\AffairsController@exportCourseWeightsCohortPdf	Auth + role: affairs,admin

Method	Path	Handler	Access
GET	/api/affairs/course-weights/export-student-pdf	Api\AffairsController@exportCourseWeightsStudentPdf	Auth + role: affairs,admin
POST	/api/affairs/course-weights/student-decision	Api\AffairsController@updateStudentAcademicDecision	Auth + role: affairs,admin
GET	/api/affairs/dashboard	Api\AffairsController@getDashboardStats	Auth + role: affairs,admin
GET	/api/affairs/leaves	Api\AffairsController@listLeaves	Auth + role: affairs,admin
POST	/api/affairs/leaves/{id}/status	Api\AffairsController@updateLeaveStatus	Auth + role: affairs,admin
GET	/api/affairs/messages	Api\AffairsController@listMessages	Auth + role: affairs,admin
POST	/api/affairs/messages	Api\AffairsController@sendMessage	Auth + role: affairs,admin
GET	/api/affairs/messages/conversation/{userId}	Api\AffairsController@getConversation	Auth + role: affairs,admin
GET	/api/affairs/metadata	Api\AffairsController@getMetadata	Auth + role: affairs,admin
GET	/api/affairs/notifications	Api\AffairsController@listNotifications	Auth + role: affairs,admin
POST	/api/affairs/notifications/read-all	Api\AffairsController@markAllNotificationsRead	Auth + role: affairs,admin
POST	/api/affairs/notifications/{id}/read	Api\AffairsController@markNotificationRead	Auth + role: affairs,admin
POST	/api/affairs/parents-students/create-parent	Api\AffairsController@createParentAndLink	Auth + role: affairs,admin
POST	/api/affairs/parents-students/link	Api\AffairsController@linkStudentToParent	Auth + role: affairs,admin
GET	/api/affairs/parents-students/parents	Api\AffairsController@listParentsWithStudents	Auth + role: affairs,admin
POST	/api/affairs/parents-students/unlink	Api\AffairsController@unlinkStudentFromParent	Auth + role: affairs,admin
GET	/api/affairs/parents-students/unlinked	Api\AffairsController@listUnlinkedStudents	Auth + role: affairs,admin
GET	/api/affairs/pending-accounts	Api\AffairsController@pendingAccounts	Auth + role: affairs,admin
GET	/api/affairs/photo-change-requests	Api\AffairsController@listPhotoChangeRequests	Auth + role: affairs,admin
POST	/api/affairs/photo-change-requests/{id}/approve	Api\AffairsController@approvePhotoChange	Auth + role: affairs,admin
POST	/api/affairs/photo-change-requests/{id}/reject	Api\AffairsController@rejectPhotoChange	Auth + role: affairs,admin
GET	/api/affairs/profile	Api\AffairsController@getProfile	Auth + role: affairs,admin

Method	Path	Handler	Access
POST	/api/affairs/profile/password	Api\AffairsController@updatePassword	Auth + role: affairs,admin
POST	/api/affairs/profile/update	Api\AffairsController@updateProfile	Auth + role: affairs,admin
GET	/api/affairs/semesters	Api\AffairsController@listSemesters	Auth + role: affairs,admin
POST	/api/affairs/semesters/{id}/activate	Api\AffairsController@activateSemester	Auth + role: affairs,admin
GET	/api/affairs/student-service-requests	Api\AffairsController@listStudentRequests	Auth + role: affairs,admin
POST	/api/affairs/student-service-requests/{id}/process	Api\AffairsController@processStudentRequest	Auth + role: affairs,admin
GET	/api/affairs/student-services	Api\AffairsController@getStudentServices	Auth + role: affairs,admin
POST	/api/affairs/student-services/{id}/process	Api\AffairsController@processStudentService	Auth + role: affairs,admin
GET	/api/affairs/students-academic	Api\AffairsController@getFilteredStudentsForAcademicCard	Auth + role: affairs,admin
POST	/api/affairs/students/promote	Api\AffairsController@promoteStudentsYear	Auth + role: affairs,admin
POST	/api/affairs/students/{id}/reset-device	Api\AffairsController@resetDevice	Auth + role: affairs,admin
GET	/api/affairs/university-ids	Api\AffairsController@listUniversityIds	Auth + role: affairs,admin
POST	/api/affairs/university-ids	Api\AffairsController@addUniversityId	Auth + role: affairs,admin
GET	/api/affairs/university-ids/next-id	Api\AffairsController@nextUniversityId	Auth + role: affairs,admin
DELETE	/api/affairs/university-ids/{id}	Api\AffairsController@deleteUniversityId	Auth + role: affairs,admin
POST	/api/affairs/university-ids/{id}/update	Api\AffairsController@updateUniversityId	Auth + role: affairs,admin